BABEŞ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE
SPECIALIZATION COMPUTER SCIENCE IN ENGLISH

DIPLOMA THESIS

CData: a small, low-overhead data-oriented C superset

Supervisor
Assoc. Prof. PhD. Grigoreta Cojocar

Author
Manta Andrei Dragoș

2026

ABSTRACT

Data oriented design is the idea that the fundamental goal of programs is to manipulate data, and, as such, data, not abstractions, should be the focus of programmers engineering solutions.

In this work, a history of programming languages emphasizes the ways software engineers have become more and more estranged from the physical nature of data, and the positive and negative aspects of programming language abstractions are argued from the perspective of runtime efficiency.

A small superset of C called CData is proposed to help bring attention back to the data programs handle. The language functions by way of transpilation into standard C then compilation into executables and it is specified both directly and through analysis of full transpilation examples of its major features - the usage of the structure of arrays (SOA) aggregate type in contrast to C's array of structures (AOS) pattern, and hot-cold splitting of data in C structs.

The build process put together for the purposes of this work follows a classic pattern - the CData source code is lexed, the output representation is fed into a parser that constructs a syntax tree (ST) representing the concrete program, the ST is fed into the transpiler that parses it to produce C code equivalent to CData, according to CData's specification and the transpilation rules.

The transpiler application is wrapped in a minimal code editor, that allows for side-by-side visualisation of the original CData source code and its C resulting transpiled code.

Contents

1 Introduction **1**

2 A brief history of hardware and abstraction **3**
2.1 The memory hierarchy 4
2.2 Caching 8
2.3 The beginning of software abstraction 11
2.4 The C programming language 12
2.5 Object-oriented programming 14
2.6 A modern spin on low-level interaction 16

3 The CData programming language **19**
3.1 The effectiveness of memory reordering 19
3.2 Data-oriented programming 20
3.3 The framework of CData 21
3.4 The static soa type in CData 23
3.5 A complex static soa example 27
3.6 Pointers to soa types 32
3.7 Hot-cold splitting 34

4 The design and implementation of the application **39**
4.1 The creation of the transpiler 39
4.2 The creation of the text editor 40

5 Testing and results **42**
5.1 The testing setup 42
5.2 The search space 43
5.3 Test 1 - full CData utilization versus C 44
5.4 Test 2 - tweaking hot-cold parameters 44
5.5 Test 3 - CData soa utilization versus C 47
5.6 Comparison to other DOD tests 47

6 Conclusions **49**

Bibliography **50**

Chapter 1

Introduction

This work is meant as an exploration of low-level data representation and a presentation of a possible alternative to current norms: the CData programming language, a superset of the C programming language that is aimed at morphing classic C memory patterns, such as the struct and the array of structs (AOS), into data-oriented memory patterns, such as the struct of arrays (SOA), while adding as little overhead as possible and maintaining C's simple semantics.

The reason CData was developend and deemed worthy as the subject of the present work is that exploring ways to improve an already popular solution, the C language, is quite promising in terms of expected program speedup. While speeding up individual applications or algorithms can bring large performance gains for their particular use cases, directly providing general speedup solutions for the entire underlying language can be more impactful.

The field of data-oriented programming (DOD), is still both rather young and less developed than modern staples of software, such as the object oriented model. As such, a large part of the literature concerning the ideas and developments in the field are either coming out directly from engineering companies whose engineers are tasked with producing said advances (much like C and C++ were results of the research department at Bell Labs in the 1970's), or are published in gray literature - online blogs, informal software conferences and the like. Nyberg proposes and demonstrates a deep step by step iteration of mutating polymorphic code into entity-component-system code, thus, the staple of OOP programming into the leading system of DOD programming, and explains the advantages and disadvantages this might incur [Nyb21]. Pellacini et al. explain the implementation of a custom 3-dimensional rendering system, and, although the performance and usability they describe are positives, the most impressive aspect is the visual fidelity of the images it managed to produce, on par with state-of-the-art graphics engines [FP19]. Fabian provides a detailed description of the state of data-oriented design and the ideas

that helped shape it thus far, describing deep DOD subjects such as treating memory objects as database entries and applying appropriate relational normalization directly to them [Fab18] . Acton's work was a presentation held at a gaming development conference and is considered one of the founding acts of modern DOD thinking, both by the detailed technical documentation it provided and by the software philosophy ideas it introduced, especially in opposition to OOP philosophy [Act]. Bayliss's work is a summary and introduction to the Unity Engine DOTS (data-oriented technology stack), one of the largest implementations of the DOD concept in modern mainstream software technology [Bay22].

In Chapter 2, a brief history of abstraction and programming language semantics will be given, but also a description of how these semantics enforce certain use patterns on modern hardware that sometimes causes it to be underutilized.

In Chapter 3, the CData programming language will be described in detail, that is, the few additions it brings on top of classic C will be exemplified and the motivation for their introduction will be explained. As well, the implementation details of a transpiler that transforms CData code into classic C code that can be readily compiled are given, with the goals of having the system be as clear, concrete and reproducible as possible.

Chapter 4 presents a series of tests that were performed on the same program, written alternatively in CData and classic C, and the results of these are presented, interpreted relative to what the theoretical implementations predicted, and compared to similar tests performed in the field of data-oriented programming.

Chapter 5 presents the conclusions of the paper and indicates further directions the semantics prescribed for CData might evolve towards after proper exploration thereof.

Generative AI usage

Generative AI was used in this work in order to produce some figures herein. Except otherwise mentioned, the presented figures were generated by AI, with close supervision and oversight by the author, and the information and structure therein are correct and in accordance to the author's intentions.

Chapter 2

A brief history of hardware and abstraction

In this chapter, a brief history of the evolution of abstraction in programming languages is provided, emphasizing the tradeoffs that high-level languages have imposed over time and, along with enormous ease-of-use aspects they introduced, some capabilities programmers lost by widely adopting them, with the purpose of serving as a motivation for the methods described in later chapters. Firstly though, a description of the relevant underlying hardware is necessary to properly understand the source of many programming issues.

In the early stages of fully electronic computers, in the 1940's and 1950's, one particular computer system architecture stood out and became the de facto scheme for structuring these machines ever since: the von Neumann architecture, named after Hungarian-American mathematician John von Neumann after his work on the ENIAC project that he summarized in the work "First Draft of a Report on the EDVAC" [vN]. There, he described a computer system that comprises of a central processing unit (CPU) and a, very importantly, separate Memory Unit. The central processing unit is itself composed of the Arithmetic-Logic Unit (ALU) and the Control Unit, the latter coordinating the entire operation of the machine by alternatively sending memory data to and from the ALU. This architecture was widely adopted and utilized in both early mainframe computers in the 1950's and 1960's and in the rising and powerful personal computers starting from the 1970's.

In the history of computer hardware, despite both the CPU and the memory unit being essential for a system, the development of faster, smaller and cheaper CPU's has outpaced that of memory significantly - while the number of transistors on CPU's has doubled every two years (a phenomenon hypothesized by Gordon Moore, one of the founders of Intel, and synthesized as Moore's law) from the 1960's until recently, thus increasing processing speed exponentially over time, the size and speed of memory units have only increased linearly, hence a significant

gap has emerged as a large bottleneck in computer performance due to the processor waiting significant amounts of time (a state called stalling) for the memory to provide requested data [O'R16].

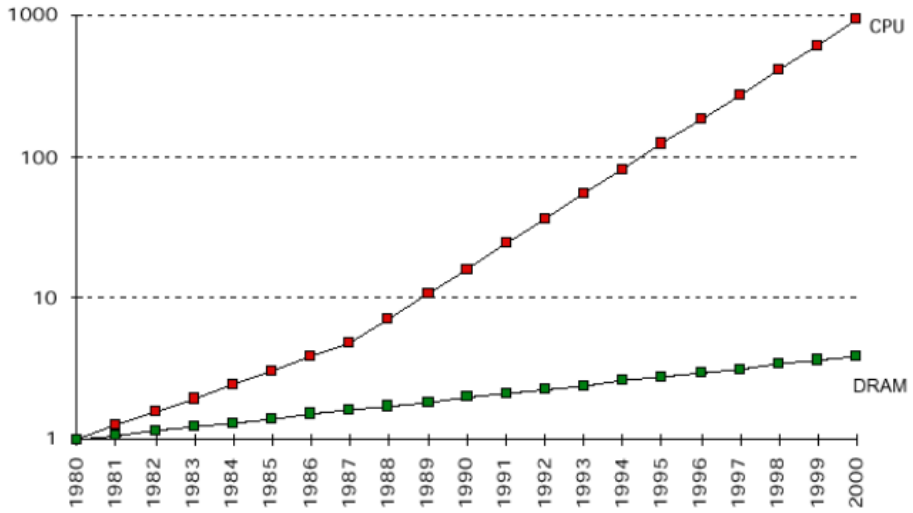


Figure 2.1: The processor-memory performance gap, tracked since 1980.

The processor-memory performance gap is not only increasing, the rate at which it's increasing is increasing, compounding to produce CPU's that are orders of magnitude more performant than their memory units, as shown in Figure 2.1 taken from [Car02]. One of the important reasons for this imbalance is physical - the widest-used modern memory technology, DRAM (Dynamic Random Access Memory), stores bits in electric cells that leak electric charge (hence the name dynamic) and have to be periodically recharged, causing the entire memory to be unresponsive for significant amounts of time while it's recharging. Although it is true that there exists an alternative to DRAM, SRAM (Static RAM), that does not need recharging and is always responsive, SRAM cells are significantly more complex in terms of electronic components, hence more expensive per stored bit than DRAM [O'R16]. One other unscalable wall in memory performance comes from a side effect of ever-increasing memory sizes - more memory cells implies a larger memory unit, hence electric signals must travel a larger distance within it, increasing the time until a memory request can be completed [O'R16][PH11].

2.1 The memory hierarchy

Hardware engineers have, over the years, come up with numerous solutions to address this problem of memory performance. The widest-adopted one is the mixing of cheap DRAM with a small proportion of the expensive SRAM such that

both the larger size of DRAM and the faster response speed of SRAM may be used in unison, thus yielding the modern computer memory hierarchy, depicted in Figure 2.2. This model is composed of a head - the CPU's register memory, and successive downward layers of memory: k cache memory units, then one unit of RAM memory (called primary storage in this model) and l units of, typically, much slower memory, called secondary storage - hard disks, flash drives, the Internet and similar. There

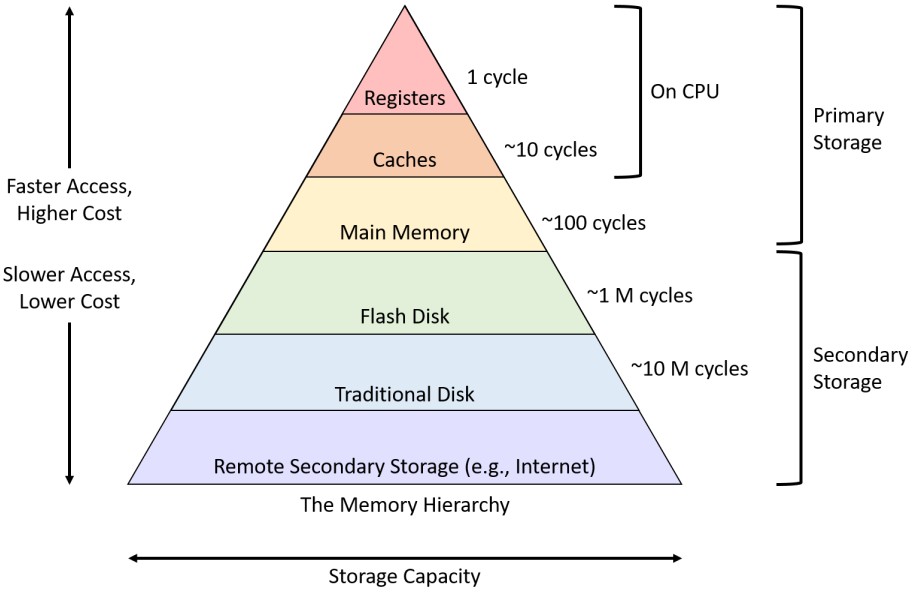


Figure 2.2: The modern memory hierarchy [MWN22]

are two metrics this model entails that give the foundation for meaningful reasoning about physical systems [Dre][PH11].

First, the cycle cost - computers are not analogic machines that run in continuous fashion, but rather digital, discrete machines that are in one particular state at a moment, then pass into a different discrete state almost instantaneously. The frequency of these state transitions is given by an electronic component called a clock, and the period between two transition signals sent out by the clock is called a clock cycle. Many engineering properties of electronic components relate to clock cycles, in particular how many clock cycles it will take a component to perform a given action. For example, it may be specified that a given processor may add two 8-bit numbers that it has stored in its internal registers in 4 clock cycles, or that the time it will take since the processor issues a memory request to RAM memory until the RAM sends back the answer to the request is 200 clock cycles. Thus, the clock cycle has become the default unit of measurement when discussing operations at the hardware level, since it gives a reasonable degree of invariance for comparing durations of execution on fixed systems and even across systems, since both CPUs and memory units have their performance currently mainly reported in number of clock cycles per second (a.k.a. clock frequency, measured in multiples of the Hertz - Hz), enabling rough

estimates to be developed - a given program that is executed in 2500 clock cycles on a given processor will execute 10% faster on a processor from the same family but with a 10% higher operating speed, or a memory request that completes in 100 cycles on a given memory unit will complete 10% faster on a unit with 10% higher operating frequency [Dre][Tan14].

Second, capacity, i.e. the maximum number of bytes a particular memory unit is able to store. Since the basic electronic component for memory storage has historically been the transistor and a transistor may be in only one of two discrete states at any time, these states are interpretable as 0 and 1 and say that one memory cell (the basic component of a memory unit) holds one bit of information at any time (the introduction of the bit as the discrete unit of measurement for information dates back to Claude Shannon in the 1930's, the founder of modern information theory) [Sha48]. Thus, the first memory units were composed of some number of bits, each of which had an address and could be read from or written to, the 0 or 1 held in each cell being then able to be interpreted as useful information by the processor during a program's runtime. It quickly became evident that working with individual bits was restrictive, and after experiments were performed on working with various numbers of bits at a time, such as 6, 8, and 10 bits, the number 8 was chosen as the fundamental unit of measurement for the memory, and, as such, each group of 8 bits was called a byte, each byte having its own address and any memory access being forced to reference a grouping of 8 bits and not any individual one [Tan14]. Apart from clock frequency, an essential characteristic of memory units is their size, given in multiples of bytes, as large an amount of bytes as possible being very desirable for such a unit [O'R16].

These two metrics provide a relation pertaining to the memory hierarchy - the raw access speed and the storage speed move in opposite directions as we go down the hierarchy. A fictitious, but perfectly reasonable example, is not difficult to construct in today's hardware landscape. Assume there is a computer system with a CPU that has a single core containing a single ALU and some registers - fast but very small memory units that are placed very close physically to the ALU, and, as such, can provide data to the ALU extremely quickly. This CPU also contains, a little farther from the ALU, a larger but slower memory unit called a level 1 (L1) cache. Quite farther from the ALU sits an even larger but slower level 2 (L2) cache, and outside of the CPU, orders of magnitude farther from the ALU, sits a unit of RAM, the main memory of this system. In the operating of the system, the control unit repeatedly brings instructions into the registers from the other memory levels, brings any supplementary data the instructions require from the other memory levels, sends data from the registers to the ALU, stores results from the ALU into the registers and stores back these results into the lower memory levels. The exact

mechanisms for these data transfers will be discussed in later sections. What matters as it pertains to the two metrics in question is the access speed of each memory layer, i.e. the number of cycles it costs for some data to be copied from a particular layer into a register, since the registers are the operands for most hardware instructions. This cost is in the order of 1 cycle for the registers, i.e. it takes about one CPU cycle to copy the information from one register to another, in the order of 10 cycles for the L1 cache, i.e. it takes around 10 cycles (note, in the context of orders of magnitude, the order of 10 cycles might mean, in practice, 20, 30 or even up to 99 cycles) to copy some data from the L1 cache into a register, in the order of 10 or 100 cycles for copying from the L2 cache and in the order of 100 cycles for copying from RAM memory. As it can be seen, the cost of accessing data from lower memory levels increases at least linearly for each level decreased. The positive aspect of successive layers is that, in each layer, the byte capacity increases - the registers in this example might be able to store 8 bytes each, and there might be a handful of them, say 20, so the total capacity is in the order of hundreds of bytes, while the L1 cache may be in the orders of thousands of bytes. The L2 cache may be in the order of hundreds of kilobytes (that is, hundreds of thousands of bytes), and the RAM may be in the order of gigabytes, or billions of bytes. As was the case for the access speed decreasing at least linearly, at least a linear increase may be observed, if not better, in the size of each downwards successive layer of the memory hierarchy. These two functions need not be inverses, but this has been the case in practice - lower levels of the memory hierarchy are larger and slower, and the converse as well. The higher speed in higher levels has been achieved both by having them be reduced in size, hence they fit close to the registers on the chips of CPUs and the signals only travel short distances, and by using the SRAM technology mentioned previously, its cost prohibitiveness being avoided by only using it for the smaller memory modules. Along with help from machine instruction set architects designing complex and large systems of hardware-level operations for memory management, these hardware implementations yielded a spectacular result - upon accessing an address of lower-level memory, the bytes around that address are also copied into higher levels of memory, and since many programming access patterns, such as container iteration, involve accesses to sequential addresses in memory, after a first access to, say, RAM, the memory region around the accessed address is copied into the L2 and perhaps L1 cache, thus the next iteration will be able to retrieve its data directly from these higher levels, yielding a maximum reduction of hundreds of cycles per memory request for the sequential-memory-access section of the program [Tan14]. This example shows that if memory were to be accessed randomly, the probabilty of subsequent accesses addressing closely-positioned locations in RAM would be decreased and the described speedup would definetly decrease. What's worse is that

there is also a supplementary cost associated to copying the entire memory region around the requested address into higher levels, cost that is paid whether or not the speedup is taken advantage of, and memory access patterns that "jump around" the memory and don't utilize caching efficiently do exist in practice, but they are not in the majority.

2.2 Caching

Most programs are composed of many sections of either code that accesses large, sequential subsections of the memory space, property called "spatial locality" of the program's data, or code that accesses a subsection of memory repeatedly in short intervals of time, property called "temporal locality" of the data. A simple example of spatial locality is that of scanning an array in search for a value, an operation ubiquitous both in low-level programming and in significantly more abstract systems such as databases, in which successive memory locations are loaded from the main memory into registers and compared to some value, and an example of temporal locality is the recursive function call - the data representing the instructions making up a function is loaded into the execution section of the CPU over and over until the end of recursion is reached. Let us perform a rough calculation of the execution times of these two memory access patterns using real hardware data, in order to properly gauge the speedup that memory layering might provide. Assume in use there is a Kaby Lake i7–7660U CPU from Intel, which contains multiple execution cores and is significantly more complex than our model of the hierarchical memory, but it is possible to only use one core at a time, so this assumption is reasonable. Each core contains an L1 cache of 32 kilobytes (KB) that takes 5 cycles to be accessed, an L2 cache of 256 KB that takes 12 cycles to be accessed and there is also an L3 cache of 4 megabytes (MB) shared by all cores and only utilized by one in our example that takes 42 cycles to be accessed.[Intb][Fog] Irrespective of the manufacturer of the RAM this CPU is paired with, a cycle cost (also known as memory latency) of around 100 cycles for one round-trip request is to be expected.[Tan14][Dre]

Thus, for the first access case, in RAM, an array of particles is stored. Each particle is comprised of three numbers, say three integers, representing the speed of the particle in the three spatial dimensions, and most systems today store an integer on 4 bytes, thus one particle will take up 3 * 4 = 12 contiguous bytes in RAM. Assume there are $2 \wedge 30$ particles, taking up 12 * $2 \wedge 30$ = 12 gigabytes(GB) of contiguous memory. In order to iterate over the entire sequence of particles and find the highest speed of any particle, hence only memory reads are required (except for the memory write operation involved for storing each new maximum found, but that cost can be ignored). Let us assume the next model for memory caching: the sizes of the

caches are known, thus it is known that L2 is 8 times bigger than L1, and L3 is 16 times bigger than L2; it can be said that, if the processor requests a memory value and it is not stored in L1, if it is present in L2 then that value and all values to the right until 32 KB get copied into L1; if a value is searched for in L2 and not found, if it is present in L3 then that value and all values to the right until 256 KB get copied into L2 then that value and all values to the right until 32 KB get copied into L1; if a value is searched for in L3 and not found, then it must be in RAM, it is copied from RAM into L3 along with all values to the right until 4 MB, it is copied into L2 along with all values to the right until 256 KB and it is copied into L1 along with all values to the right until 32 KB. This is an example of an inclusive cache policy, since multiple levels of the cache store the same data at the same time. Now, the requests to memory by the CPU will be: provide the data from bytes n up to n + 3, n + 4 up to n + 7 and n + 8 up to n + 11 (it was mentioned an integer is stored on 4 bytes and there are 3 of them per particle, and all three of them are needed to perform the calculation for the speed of a particle). Assume n is 0 (i.e. the array starts at address 0 in RAM) - the first memory request, that for bytes 0 1 2 3, will not find this data in L1 or L2 or L3, but only in the RAM, from where it gets copied along with data until the address 4 * 2 ∧ 20, i.e. 4 MB, into L3, along with data until the address 256 * 2 ∧ 10, i.e. 256 KB, into L2, and along with data until the address 32 * 2 ∧ 10, i.e. 10 KB, into L1. Hence, the cycle cost for the first memory access is 100 cycles (accessing RAM). It is true that the many other copy operations performed here into L1, L2 and L3 have a cycle cost but that can be ignored for now. Now, the second memory access will ask for the second integer of the first particle in the array, i.e. the bytes at locations 4 5 6 7, but these bytes are already in L1, hence the cost is only 5 cycles. This will also be the cost of the next 32 KB / 4 B - 1 = 8091 requests, since all that data to the right of the data at address 0 fit in L1. Upon requesting the address 32 KB itself, i.e. the first address that did not fit in L1, it is looked for in L2, it is found there and the data from there up to address 32 KB + 32 KB - 1 will be copied into L1. This access will cost 12 cycles, but the next 32 KB / 4 B - 1 accesses will only cost 5 cycles since the data was copied into L1. This pattern will continue until the request of address 256 KB, the first address that did not fit in L2, which will be found in L3 and the data from 256 KB up to 256 KB + 256 KB - 1 will be copied into L2 and up to 256 KB + 32 KB - 1 into L1. This access will cost 42 cycles, but it will allow for the next 256 KB / 4 B - 1 accesses to cost either 5 cycles or 12 cycles (1 in every 32 KB / 4 B accesses). The same pattern holds for L3 - every 4 MB a request is made that cannot be fulfilled by L3, 4 MB of data is copied from RAM into L3, 256 KB into L2, 32 KB into L1, and the access now cost 100 cycles. It is now apparent that, for every batch of 4 MB accesses, there is one access that costs 100 cycles, the one that goes all the way to RAM, (4 MB / 4 B) / 256 KB - 1 = 3 accesses that cost 42 cycles, the ones

going to L3, (4 MB / 4 B) / 32 KB - 1 - 3 = 28 accesses that cost 12 cycles, the ones going to L2, and 4 MB / 4 B - 1 - 3 - 28 accesses that cost 5 cycles, the ones going to L1, which is apparent is the vast majority of accesses, hence the full memory-related cycle cost of the given program is (5 * (4 MB / 4 B - 1 - 3 - 28) + 12 * 28 + 42 * 3 + 100 * 1) * 12 GB / 4 MB = 16.107.362.304, around 16 billion cycles. This is an upper bound on the real number of accesses, since the cost of copying sections from lower to higher memory levels is not negligible, but the estimate is still meaningful. If this tiered memory was to not be in place and every request was to go out to RAM, the cycle cost would have been 100 * 12 GB / 4 B = 322.122.547.200, more than 300 billion cycles, thus 20 times more [Tan14][Dre]. This example is illustrated in Figure 2.3.

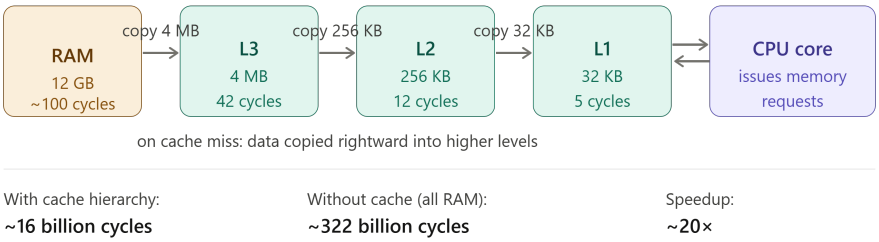


Figure 2.3: Cache operations in the concrete case.

For the second access case, let us assume the code for some recursive function takes up 8 KB of memory, a reasonable estimate, even a little on the large side. In this case, except for the first access that goes all the way to RAM and costs 100 cycles, all the code will fit in the L1 cache and all accesses will cost 5 cycles instead of the 100 it would have taken without the memory hierarchy, again resulting in a roughly 20 times speedup. It is the case in modern systems that the code of a program is usually orders of magnitude smaller than the data that program makes use of, and oftentimes the entire code section fits in the caches, this being part of the reason why hardware designers currently implement L1 as two physical caches - L1i just for instructions and L1d just for non-instructions, i.e. data, the result being that code remains in L1i for long periods without being overwritten, all the while providing useful information.[Tan14][Dre]

Real life systems are more complex: they do not operate on single cores, they have multiple cores that usually share the L3 cache, but not L1 and L2, those being per-core caches, but since memory must logically look the same for all cores, if one

of them performs a write to data in its cache, that data must be invalidated in the other cores' caches, procedure which takes a non-negligible amount of cycles (the architecture that performs this is called MESI) [UoP]; the model that copies over arbitrary amounts of bytes between levels, which is not the case in actual systems, usually the amount that is passed is 64 bytes; it was also assumed here that, upon a "cache miss" (data requested by the processor not being present in the level it was requested from), there is a motion recursively down the memory hierarchy and fill higher levels with subsections of lower levels, and that is not the case, the policies for filling caches being far more complex (the discussion focused on an inclusive policy, but there exist non-inclusive policies where data may or may not be present in multiple levels and also exclusive policies where data must only be present in one level at one time); only memory reads were performed, whereas actual programs also perform numerous writes, writes which either must pass directly all the way to RAM (the write-through policy) or which are only kept in the caches, but the written to data must be marked as dirty and extra bookkeeping must be performed to ensure RAM is also synchronized at some later point (the write-back policy). As a last point, modern hardware is designed to work in conjunction with operating systems and vice versa, hence current processors, more specifically their memory management unit (MMU), implements paged and segmented memory, processes by which addresses the processor requests are treated as virtual addresses and translated to physical memory addresses through another system of indirections and caches, thus one single memory access may take significantly more time than the raw speed of accessing the given memory layer, since indirections add overhead [Tan14][O'R16].

2.3 The beginning of software abstraction

The advances of the hardware were discussed from its most primitive state up to modern systems and exemplified its operations, now the development of software techniques can be brought forth to better utilize hardware resources.

In the 1950's, computers were programs directly in machine language - the hardware manufacturers provided the set of instructions the CPU recognized and their precise effects, thus it was straightforward to design algorithms that used machine instructions to perform a task - decide the sizes and types of operands, look up in the table of instructions the one that matched the sizes and performed what the algorithm implied, then write the byte sequence that encoded that instruction and those operands [O'R16]. The downside was that the instruction tables were large, hence time consuming to look into by hand for each instruction, and each single instruction only progressed the program state by a small amount. The innovation came with the advent of the first assembly languages, which mapped symbolic in-

struction codes (mov, add, sub and so on) to byte sequences, hence the assembler program was able to automatically encode the logical operation programmers provided it into a sequence of bytes all by itself - if it was provided ”add par1 par2” with both parameters sized to 2 bytes, its internal logic and knowledge base was sufficient to produce the byte sequence for the add hardware instruction with parameters of that size, while for a programmer trying to produce this sequence by hand it had proved tedious - if the add instruction supported 4 different sizes of operands and each operand might have been a register, a constant value or a memory location the table would already be 4 * the number of registers * the number of possible ways to specify constants and memory addresses, and with each addition to the capabilities of the add instruction this table would grow by a multiplication factor, not by adding a constant number of rows. It is thus clear that assemblers provided programmers with reliable ways of writing longer and more complex programs, but this came at a cost - programmers were less aware of the actual ways in which logical instructions were implemented in the hardware, and lost a small part of their power to reason about the exact state of the program and its memory space, since they were no longer familiar with the actual encoding tables and the ad-hoc (and oftentimes counterintuitive and nonuniform) compromises hardware designers had done to be able to increase the hardware’s performance [Inta].

2.4 The C programming language

Around 1970, the C programming language was introduced, after years of experimentation and development. It was a large innovation and, although not the first high-level language, it quickly gained its place as the first largely-adopted and the lowest-level high-level programming language. From the perspective of abstraction, the most important concepts it solidified in mainstream programming are data types, scopes and modules [KR78].

Data types abstract away the necessity of tracking individual bytes when working with multibyte logical structures - it is significantly easier to declare a variable of a given data type and use the operators defined for that type on that variable and have the C compiler translate all these operations into assembly instructions, taking away the need for the programmer to keep in mind particularities of instructions and the manipulation of their operands for various sizes, endian-ness and so on.

Scopes (or blocks) were introduced as a way to separate the logical space of a C program - variables may be in the global scope, in which case they may be accessed from within any other scope which is useful for data that is truly meant to be needed across the entire program, or they may be in the local scope, i.e. within a function, and they are accessible only from within that function, thus enforcing the

mental model that states that each piece of the program, in this case the pieces of C programs being functions, should do one thing and do it well - they should declare and use their own variables which are meaningless to the rest of the program.

Modules work in a similar direction with scopes - by splitting programs in header and implementation files, the headers serve as proto-interfaces that expose only the signatures of functions within the modules and indicate that users of said modules should base their applications on those signatures and not on the implementations of the functions [KR78].

With each abstraction come costs however, mainly related to offloading cognitive effort onto computer systems. Modules allow programmers to not worry about the implementation of functions they use, but they also make them less aware of the choices in the implementation that shape the runtime and performance of their program, problem that only compounds as programs have more and more such dependencies. Scopes both help the program be more structured, but also enforce a given order of execution - function A is called and gets its own stack frame for local data, it calls function B that receives the same treatment, it calls function C and so on, and when it comes to returning the order is thus forced to be C returns then B returns then A returns, hence if something anomalous happens somewhere down the line, say in C or lower, the program must return back through the stack in one exact order, usually by having functions return special values interpreted as error codes, which can prove an awkward and difficult to use system for handling exceptional situations, compared to non-scoped languages that may have the ability to have the program jump around the code arbitrarily, say to an error handler directly. The largest tradeoff, at least for this thesis' purposes, comes from the introduction of data types - though convenient to use (numerous bugs in assembly programs appearing from blunders in interpreting the proper positions of individual bytes within large structures), they add limitations - variables of data types are most often used in conjunction in groupings C calls structs, this being an important part of the C memory model - a struct is a pseudo-contiguous group of bytes corresponding to the variables declared for that struct, and the compiler most often adds padding bytes between the actual bytes of the variables in order to align them to memory addresses that are multiples of the variable byte size, a condition imposed by the hardware for efficient retrieval of multibyte constructs. This padding would have been avoided by manually selecting the byte placements in memory, however compilers most often are not allowed to analyse and optimize byte structures because it would break compatibility between systems with different compilers and it would cause the programmer to lose the ability to reason about the structure of memory since they wouldn't be able to accurately infer byte orders within structs, this being the reason why the C standard disallows struct reorderings. As such, sometimes

surprizing amounts of memory are wasted due to *padding* within structs - assume we declare a `struct A` with members `char b` and `int c` in this order, the compiler on a 64-bit system would most likely produce a struct that occupies 8 bytes of space - the char takes up 1 byte, the int takes up 4 bytes but is not aligned within the struct to a memory address that is a multiple of 4, hence 3 bytes of padding will be added between the char and the int, thus causing a struct of 8 bytes to only contain 5 bytes of actual data.[KR78] Such issues apply when these structures are stored in arrays and cause large areas of working memory to be wasted on padding, a problem which the methods described in later chapters do address.

2.5 Object-oriented programming

In 1985, Bjarne Stroustroup introduced an extension to the C programming language - C++, or C with classes [Str]. The class concept, although having existed for some time but being truly made a staple of mainstream programming by C++, was meant to serve as the basis for a new paradigm - object-oriented programming, or OOP, objects being concrete instances of classes and classes being an extension of data types. This new style of creating programs was focused on coding the lifetime and interactions of objects, thus each objects, as a member of a class, behaved necessarily by the rules prescribed for that class, with class-specific variables, referred to as fields, and class-specific behaviours, referred to as class methods, i.e. functions that could only be called through an object of the class. In this way, the attempt was made to synchronize programs with the real world, by assimilating code objects and classes to real objects and types of objects and class behaviours to real interactions between objects. C++ brought forward a significant amount of abstraction, mainly coming from three sources - object management, class hierarchies and generic programming.

An object, in the C++ way of thinking, is a concrete instance of a class - the class is the blueprint by which the compiler writes the instructions for composing the physical object in the memory space. As such, any class requires two functions - the constructor and the destructor. The constructor ensures that the memory the object occupies is not only allocated, but also filled with the bytes that can be interpreted as an object of the class type, and the destructor ensures that memory, as well as any memory referenced therein through pointers, is properly deallocated and freed. A large upside to this is that memory is formatted in accordance to the structures of a class without requiring manual intervention from the programmer, with the coupled downside that constructor and destructor calls are often hidden by having the compiler only insert them during compilation, thus lowering the programmer's ability to easily reason about the exact structure of the call stack at any moment.

Class hierarchies are an enormous paradigm shift in programming - classes may be now be formed into trees, each child class both inheriting the fields and behaviour of the parent class and being able to add fields and behaviours alongside or overriding those of the parent. Thus, a guarantee is offered that a child class will be able to be used in any situation a class higher in its hierarchy may be, since it necessarily has at least the same fields and method names, the classic example for this being that class `Animal` may have a `string fur` field and a `MakeSound()` method, then its child class `Dog` will have the `string fur` field and the `MakeSound()` method, and wherever an `Animal` object is expected, substitution with a `Dog` object will produce reasonable results. This effect is achieved by having every child class literally have as fields every single field of the parent in the same order in memory, and for methods by having each class possess a hidden virtual pointer (vptr) to a table (vtable) of methods for that specific class and either filling in that table with the methods of the class if they override the parent behaviour or the methods of the parent class if they do not. This example is illustrated in Figure 2.4.

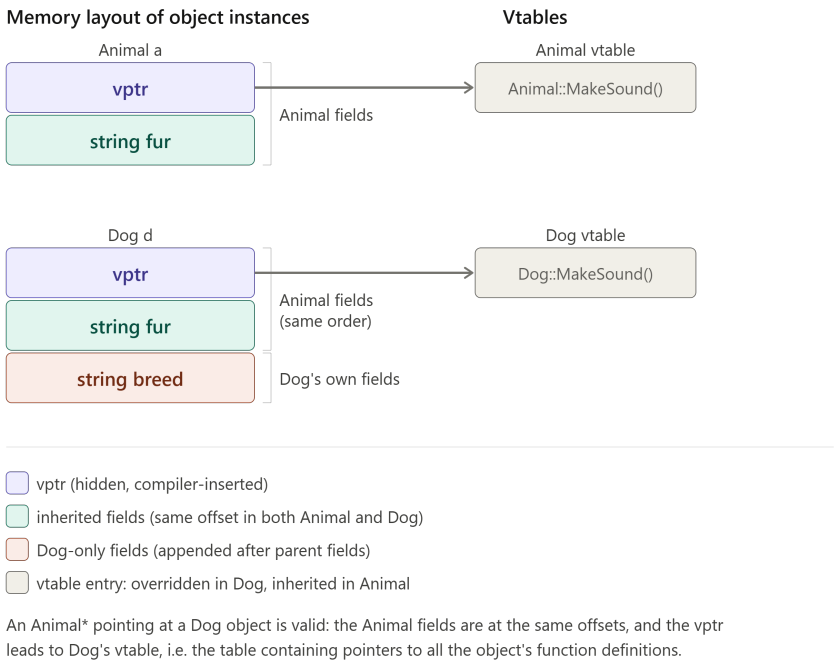


Figure 2.4: Memory and vpointer layouts in the concrete case.

This behaviour - a class being a legal substitute for any class above it in the class hierarchy, is called polymorphism and it both allows for ease of use when modeling complex real life systems, but it also introduces significant amounts of inefficiency - if a function exists both in a class and in classes higher in the hierarchy, its call on an object of the class must follow an indirection into a method table, and this effect must happen at every call; more over, when an object is instantiated in memory, all

the constructors of all classes higher in the hierarchy must be called in order to properly initialize the object, and the same happens for the object's destruction and the (possibly numerous) destructors of higher classes. This forces large recursion depths for instantiating objects deep in the class hierarchy, and makes it such that the stack contains many pieces of information for the bookkeeping of stack frames instead of useful information, it being known that stack overflow is a real and serious issue even in modern systems. A second efficiency issue caused by objects containing all fields of their higher classes is that much memory often goes to waste - an object that is 10 layers down in the hierarchy may contain 100 fields totaling hundreds of bytes but maybe only uses the fields declared in its own class and a few classes above, thus wasting large portions of memory, even though the class hierarchy implies almost no actual runtime overhead for the bookkeeping of memory. One way this problem may be alleviated that has been tried in modern programming language implementations has been the Entity Component System, the innovation here being that each "class" (rather pseudo-class, more similar to C structs than C++ classes) may choose the fields it contains, thus avoiding wasting bytes on never-used fields, but this breaks the classic object-oriented class hierarchy completely and is not compatible with it as such [Str].

The third and possibly most difficult to learn to use properly abstraction introduced in C++ is generic programming, i.e. designing methods to be able to work with parameters of any data type, this introducing both flexibility - a method logically usable on many types, such as a comparator, may only be written once and applied anywhere, but also opacity and unpredictability - it may be difficult to reason about the implicit decision the compiler will make in trying to have the method be compatible with the particular types the programmer provides in various places, adding yet another layer of uncertainty and offloading cognitive pressure onto the automatic system while reducing knowledge about the internal structure of programs.

2.6 A modern spin on low-level interaction

In modern years, many high-level programming languages have been created, high-level even in comparison to C++, the biggest abstractions added after C++ being, on the one hand, reference-only memory access, by which raw bytes could no longer manipulated as C and C++ allowed, but only variables may be modified through statements acting on them, practically changing the smallest interactable unit in programs from the byte to the primitive data types, and on the other hand, loosely typed languages, in which variables need not have unchanging data types, but may change data type during their lifetime depending on the need for them to

be reinterpreted by operations acting on them. The languages that best emphasize these two abstractions are Java, in the case of reference-only memory access, and Python, in the case of loosely-typed languages, but the modern language most fitting for being treated at the end of this chapter is one currently in development and used by a handful of people and companies - the Odin programming language[Hal], which is meant to alleviate some issues with classic C programming and help programmers better utilize the hardware available to them. One main feature of Odin that differentiates it from C is the use of the structure of arrays, or SOA pattern. In usual C programming, structs are declared and used as containers for individual data pertaining to some concept modeled by the struct, for example a particle may be modeled by a struct with four float fields, the first three representing its velocity in the three spatial axes and the fourth representing its mass, and one char field at the beginning representing a code that denotes what kind of particle each individual one belongs to. If one wishes to keep data about numerous such particles, one may declare a variable they name ”particles” that has the type struct particle[n], i.e. it is an array of n particle structs, a contiguous region of memory where each particle struct is laid out one after another and represents the most primitive form of a data container. As discussed earlier, structs in this array may, and in this example actually do, contain padding bytes - the four floats occupy four bytes each and must be aligned to start at an address multiple of four with respect to the beginning of the struct, but the char only occupies one, hence the compiler adds three padding bytes between the char on the first byte and the float starting on the fourth byte in order to keep the struct properly aligned. It is clear, then, that each out of the n structs in the array contains 3 bytes of useless padding, thus 3 / (1 + 3 + 4 * 4) = 15% of the entire array is wasted in this memory scheme. On the other hand, the Odin SOA pattern dictates that arrays of structs may be reordered in order to reduce the wasted space, thus: the variable particles of type array of size n of struct particle, struct particle containing, say, fields of type float x, y, z, m and field c of type char, may be transmuted into variable particles’ of type struct soa-particle (not an array type anymore), containing fields x’, y’, z’ and m’ of type array of size n of float, and field c’ of type array of size n of char, the code for this comparison being that in Listings 2.1 and 2.2, respectively.

Listing 2.1: normal C struct

```
struct particle {
    char  c;
    /* 3 bytes of padding */
    float x;
    float y;
    float z;
    float m;
};
struct particle particles[n];
```

Listing 2.2: soa C struct

```
struct soa-particle {
    char  c[n];
    float x[n];
    float y[n];
    float z[n];
    float m[n];
};
struct soa-particle particles';
```

Thus, the SOA-style struct contains the same amount of information as the old one, but analysing the padding one may see that there is no need to add padding inside any of the inner arrays of soa-particle, since they are arrays of primitive type and by definition properly aligned, and there may only be the need to add padding after the array of n chars in order to ensure the first array of n floats, and indirectly the other three as well, starts at an address multiple of the size of a float. By this simple rearrangement we managed to cut the 15% losses of the classic C scheme, although we also introduced the need for new access patterns - the access particles[0].c in the old struct must now be particles.c[0], and while it is trivial to change between the two styles, the fact that existing C code needs rewriting and will not work by only changing the struct itself is a problem for implementing such a design choice. This example simply illustrates the mechanism Odin uses for converting between classic (AOS - array of structs) and SOA memory patterns, the actual syntax is different but not in meaningful ways. What is meaningful is that in Odin the only necessary change from changing an AOS to a SOA is to add the #soa decorator next to the data type of the desired variable. One important limitation that will be heavily expanded upon in later sections is that this transformation is 1-level deep and non-inner-struct only, i.e. only fields that are not structs are transformed from simple types into arrays of those types, for example `struct A  struct B b; int c;;` will, upon being decorated with #soa while in an array configuration - `var_name:  #soa[n]A`, transform into a struct that contains an array of n ints and, crucially, an array of n `struct B`'s that remain in the configuration B was declared as, the #soa transformation did not apply recursively, Odin forcing programmers to also denote the `b` field of struct as #soa if they desire that field to be rearranged as well [Hal].

Chapter 3

The CData programming language

In this chapter, both the reasons for introducing the changes represented by CData will be presented, and concrete implementation examples will be given and thoroughly explained.

3.1 The effectiveness of memory reordering

Computer programs, at least in the imperative programming paradigm, the dominant paradigm in modern software, are sequences of instructions that operate on the program's data. This is both the simplest, but at the same time a very powerful observation - no matter the programming language or goal of the given program, it is a sequence, most likely a branching sequence, of statements that mutate, add or remove data from the program's memory space, data which is then interpreted, either by other programs or by peripheral devices that translate it into a human-parsable form. As such, it is reasonable to assume that the structure and usage patterns of the data are important for the efficiency of the program[Fab18][Act], and, indeed, in practice, many of the bottlenecks in modern programs arise from problems with how data is accessed: enterprise level software depends on databases of gigantic size that are often distributed across the world in servers, scientific computing systems must perform innumerable calculations to achieve precision and maintain coherence relative to the physical systems they model, and AI models have arrived at billions of parameters in order to produce desired outcomes - all of these applications touch enormous amounts of data, and, given the slow development of memory unit speeds when compared to CPU speeds presented in Chapter 2, programs are often left hanging while waiting for memory to provide data they require. As was exemplified in Chapter 2, changes as simple as keeping the data packed in different patterns than what is currently the norm can and does have a large impact on actual program runtime, and, although there is vast literature on this phrase, caching mat-

ters, substantially so. While it is true that advanced algorithms and data structures are very useful and should be optimized as much as possible, this power only goes as far as the hardware is properly utilized. Given the large disparity in memory layer access times (two or even three orders of magnitude increase from registers to RAM access, considerably more if we consider hard drives as well) and given supplementary systems in modern computers, primarily the paging system, an efficient algorithm that increases the sorting speed of an array by 5% is definitively not as important as focusing on having that array fit in a cache of a particular size, thus reducing the time of all accesses to it by, assume, 50%, implicitly increasing the speed of all algorithms performed therein as a bonus [Fab18].

While it is true that informatics is a science and should develop general methods for approaching problems, it is also true that those methods run on concrete machines, and, while some people might argue that the true importance lies in finding the mathematics and algorithms that solve problems quickly (and these people do stand on solid ground - data structures such as trees have transformed numerous O(n) operations into O(log(n)) operations, and AI heuristics have reduced search space exploration exponentially, yielding run time improvements in the order of millions or billions of percentage points) and that manipulating the methods to fit on actual machines is busywork and not worth much intellectual attention, the fact that simple changes in the memory structures of programs are able to reduce run times significantly by simply having more of the useful data closer to the processor [Bay22][Nyb21], and these improvements work out of the box for any program and any problem, even those with no closed, converging algorithms, is not something to be discarded, as will be demonstrated by the tests described in later sections. Hence, this is the reason CData was developend and deemed worthy as the subject of the present work - exploring ways to improve an already popular solution, the C language, is quite promising in terms of expected program speedup.

3.2 Data-oriented programming

This perspective, that data and data manipulation should be the main focus of programmers, and that programs are nothing but compositions of transformations of data, is known as the data-oriented programming pattern, or DOD. DOD, as a concept, has existed for decades, although it is true that, even today, it remains at the fringes of mainstream programming, mainly due to it being in opposition to OOP and rather harder to use than OOP, because it imposes a new and quite unnatural style of thinking about programs, especially for people who have been trained and have been using OOP for most of their programs. While OOP provides a simple mental framework - code objects represent concrete objects that can

be placed in abstract hierarchies, DOD forces the programmer to consider the data the program touches, the patterns it touches it in, the amounts and structure of the data, and all this usually results in complex abstract remodeling of the data based on its underlying internal connections, very similar to how relational database models provide complex algebraic operations for categorizing and operating on data.[Fab18][Nyb21]

Data-oriented programming is rather still in its infancy, academic literature on the subject still being sparse and most developments on it coming from the software engineering field and being documented in gray literature. In fact, this grey literature is what inspired the selection of DOD transformations applied to classic C - the DOD system is more popular in the video game and interactive systems fields, DOD elements being incorporated by game developers in their products for some time now and ad-hoc, without a coordinated and axiomatic view of what DOD exactly is, hence the author considered this a reasonable direction for further explorations. Despite DOD still being in its infancy, there are places where it's used extensively, such as in the Entity Component System of the Unity game engine, one of if not the most popular game engine in today's landscape, where it's used to transform native bloated objects written in OOP style into entities that choose what fields they actually need, and to create transformations that touch data efficiently in this new non-object-oriented context. Another example of DOD used in large industry projects is the use of the Odin language, which is helping to create state-of-the-art liquid and particle simulators that are in use by established video game developers [Hal].

Some principles of DOD that are common across the literature are: data is concrete and its structure is too, solutions should focus on the actual state of the data and not on writing code as generic as possible to accommodate possible future structural changes that often never happen; problems need not have a large depth of abstraction added to them, often data is able to directly express the parameters of the problem and can be directly interpreted without the need for indirections; hardware awareness is important, and the extra development time that it takes to write targeted solutions for various hardware is worth it in the long term since it produces software that will run better every time it is used.[Nyb21]

3.3 The framework of CData

For CData, the author chose to implement a superset of features for standard C that that adds minimal syntactic and semantic complexity. In this direction, the most important aspect of DOD from a performance improvement point of view was considered the hardware awareness idea, hence CData was centered on having programs utilize the caching system better than they currently do with the tools of C.

After reviewing the available literature for existing methods and inspiration, many interesting ideas were collected and given shape [Nyb21][Dre][Fab18]. Most of them were not implemented, either because they could not be compatibilized in a simple way with the rest of the implementation or they were considered too complex to be able to receive proper treatment, but they will be briefly described here to help shape the idea space in the author's mind, the following sections focusing on the ideas that made it into implementation and their concrete algorithms: operating system cooperation, i.e. the compiler may gather information about cache sizes, memory latencies and similar hardware parameters and modify the memory such that it fits neatly into the available space, for example by shuffling the fields in structs or data structures and even add artificial padding in order to have them easily splittable into chunks of cache-line size and thus avoid situations where components of the same logical object, e.g. the components of a node in a tree, cross over the boundary of one cache line onto the next and thus require the cache to spend two lines for processing one logical object; since, for many architectures, the length of the encoding of an operation is variable depending on the operation and operand types, e.g. for the Intel 8086 processor family instructions could have between 1 and 6 bytes, the compiler could reorder the assembly instructions and add unconditional jumps between them, in the mentioned architecture instructions not being enforced to be aligned in order to fit fully into the L1 instruction cache (that is, an instruction may start at address 63 and end at address 66, thus for a cache with line size 64 a part of the instruction will be stored on one line and the other on another line, phenomenon called cache line straddling that forces two lines to be filled up where it was not necessary to do so), so, for example, the compiler might look at an interleaved sequence of instructions of length 4, say i1, and length 6, say i2, in proportion of 1:1, so i1 i2 i1 i2 and so on, and choose to only keep the i1 instructions in one sequence that aligns exactly, say, in a cache line of 64 bytes, and keep all the i2 instructions in a sequence that also aligns exactly in a cache line of 64 bytes, a very common size in today's architectures, and add some unconditional jump instruction, say j of length 4 bytes, in the sequences in order to preserve the original order of the operations, thus resulting in sequence 1: i1 j(to the first i2 in sequence 2) i1 j(to the second i2 in sequence 2) and so on, and in sequence: i2 j(to the second i1 in sequence 1) i2 j(to the third i1 in sequence 1) and so on - the original order is preserved, some extra jump instructions were added and space was lost this way (4 bytes per jump instruction, a large amount given the number of jumps), but now sequence 1 may fit exactly in 64 bytes (8 4-byte i1 instructions interleaved with 8 4-byte jump instructions are 64 bytes) where it originally could not (the 4-byte i1 instruction and the 6-byte i2 instruction cannot be interleaved in 1:1 proportion and the result be 64 bytes, it can only be a multiple of 10 bytes), but it is true that sequence 2 now cannot align to

64 bytes so some problem persists, but it is reasonable to say that, given the many ways in which instructions of various encoding lengths appear in code sequences, a compiler could add such artificial jumps and perform reorderings such that the code segment be better aligned to hardware; a third and final example of interesting but not implemented idea was pointer discouraging - the compiler could analyse declared structs and notify the programmer when a struct contained mostly data types represented on byte sizes smaller than the pointer size for the architecture, hence keeping a pointer in said structs forced all other fields to be padded up to the size of the pointer and possibly waste much space, and while the point can be made that programmers should already be aware of padding and such effects and often have no alternative to pointers since they cannot know at compile time what amount of data they will need at runtime, it is as true that often programmers are not aware and do not take low-level details such as padding into consideration and, separately, could take actions to turn the pointerized data into statically-allocated data and thus not force small data types like chars and ints to receive, say, 7 or 4 bytes of padding up to the 8 bytes a 64-bit system pointer requires in order to guarantee alignment [Dre][Nyb21].

3.4 The static soa type in CData

One of two features that CData adds on top of classic C is the possiblity to use the Structure of Arrays pattern, in opposition to the Array of Structures, both outlined in earlier sections. SOA is also present in the Odin language and its limitations there, particularly with regard to recursive transformation, have been described previously. In the case of CData, SOA is intended to be a first-class citizen, and, as such, has received appropriate syntactic and semantic support. SOA in CData is a parallel to the struct concept, hence one may declare SOAs as blueprints for memory regions, just as in the case of structs, and the CData transpiler will utilize these blueprints and the prescribed semantics to turn CData source code into standard C code that can be passed into any C compiler and produce an executable. In comparison to Odin's SOAs, CData SOAs automatically have recursive data rewriting, which is not necessarily better or worse than Odin semantics, as we will explain later. But first, an example of the actual output of the SOA transpiler for some simple source code is analysed. Then, the general algorithm for SOA transpilation is described and then more intricate examples involving both static SOAs and the implemented SOA pointer semantics.

The syntax CData introduces is minimal: the `soa` keyword and the `@n@` pointer access. The `soa` keyword is meant as an alternative to C's struct, `soa`'s and structs may not be mixed but `soa`'s may be nested within each other for arbitrary depths.

The fields of a `soa` are specified in the same manner as the fields of a struct and are accessed almost the same, static access (with the dot notation) and array indexing being identical, access through a pointer to a non-`soa` type being identical and the only difference being in the case of a pointer to a `soa`-type where well-formed accessing requires the information of the size of that `soa`, e.g., for accessing the first element of a `soa` of size 10, `ptr@10@[0]` instead of the C `ptr[0]`.

Semantically, CData differs substantially from C, the most important discordance being that efficient memory layout are simple to write the code for but not simple to infer from the code, breaking C's strict specification that memory is laid out in the order it is prescribed in the code. The struct of arrays (SOA) mechanism implemented in CData, the counterpart to the array of structs (AOS) style found in procedural C and object-oriented C++ programming, is built such that composite types are defined via the `soa` keyword almost identically to composing through structs in C, but the transpilation mechanism produces C structs that are significantly more difficult to intuit by looking at CData source alone, due to the trees that naturally appear in struct nesting being flattened and reordered by the transpiler.

Listing 3.1: Simple CData sample

```
soa Particle{
    float x;
    float y;
};
int main(){
    soa Particle particles[100];
    particles[1].x = 1.0;
    particles[2].y = 2.0;
    particles[3].x = particles[3].y;
    return 0;
}
```

Listing 3.2: Simple CData transpiled sample

```
struct Particle_100 {
    float Particle_100_Particle_x_1[100];
    float Particle_100_Particle_y_1[100];
};
int main()
{
    struct Particle_100 particles;
    particles.Particle_100_Particle_x_1[1] = 1.0;
    particles.Particle_100_Particle_y_1[2] = 2.0;
    particles.Particle_100_Particle_x_1[3] =
```

```
    particles.Particle_100_Particle_y_1[3];
    return 0;
}
```

In Listing 3.1, a basic example of CData SOA is presented. We defined the SOA (SOA is a pseudo-data type, parallel to struct, and the intention is for SOA declarations to be as similar as possible to struct declarations and have the transpiler perform the heavy lifting, not have the programmer learn all new syntax) Particle, that we wish to contain two floats representing perhaps a particle's postion, so we write it as such. Then, we declare an array of items of the soa Particle type and work with that static array exactly as we would with classic C structs - we may index the array, causing array-to-pointer decay, and thus be able to access the individual components of each soa Particle item: `particles[1].x` represents the `x` component of the second soa Particle in the array, `particles[2].y` represents the `y` component of the third item in the array and so on, and we see this classic C-struct access pattern both on the left- and right-hand side of expressions, thus struct semantics seem to hold the same as in C. However, we should remember that that is CData source code, and CData does NOT guarantee that the order of fields in soa types is preserved as the programmer wrote it, a guarantee that is mentioned in the C specification and that CData breaks. This does not cause the transpiled code to break the specification though, hence the result is coherent from C's perspective since the C compiler only receives legal code [KR78].

In Listing 3.2, we see the transpiled C code equivalent to the CData code. The transpiler received the syntax tree produced by the CData parser for the code, then scanned it for soa definitions (these can only be found at the top-level, no inner definition is recognized). It registered the only soa definition, then scanned all the function definitions (in this case, only the main function appears) and, in the places where the soa Particle pseudo-type was used, it utilized the registered blueprint. The first line of `main()` contains a declaration of an array of type `soa Particle [100]`, thus there is a need in the final code for a struct that is able to hold `100 * sizeof(soa Particle)` bytes. `soa Particle`'s declaration only contains primitive types (it is only allowed to contain primitive types, soa types, pointers to soa types or arrays of all these three), hence its size is determinable at compile time and no recursion is needed, so `sizeof(soa Particle)` yields `sizeof(float) + sizeof(float)`, an expression we need not calculate, since the transpilation does not transmute primitive types, i.e. we had floats in the CData source, we will find floats in the C source - we only look at the size as a sanity check for the result. The transpiler will produce an internal tree representing the soa Particle definition along with the appropriate array-size information of the elements, in this case the

one in Figure 3.1:

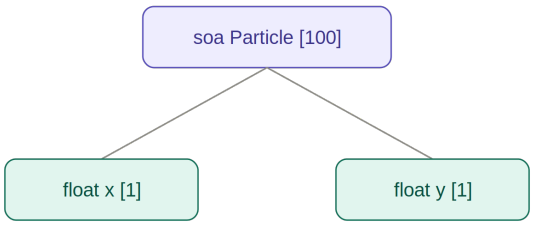


Figure 3.1: CData transpiler's tree representation of soa Particle

The transpiler will then compose the list of all leaves in inorder traversal order, in this trivial example {float x, float y}, and will create a struct of arrays (the SOA in question) of arrays of size = `the product of sizes found along the path of the tree from the root to the given node * the size of the soa array variable in the CData code`, in this case the tree path from root to `float x` being just the two nodes, `float x` having associated the array size it had in its soa declaration, i.e. 1 for a non-array type, so the final array corresponding to `float x` having size = 1 * 100 and as name it will have the path in the tree from root to it, so "Particles_100_" from the name of the struct it is in, "Particle_x_1" to denote it came from a soa declaration for a soa named `Particle`, "x" to denote it was called x in that original declaration and "1" to denote it was in an array of size 1, i.e. a bare non-array type, in that declaration. The struct name, Particles_100, symbolizes that this is the struct generated for variables that in CData were of type soa Particles [100], and any such variable from CData will have type struct Particles_100 in the C code. We see that all the information the transpiler stored, including the tree representation of the (possibly nested) soa's in the CData source, is also stored in the names of the final C fields, this feature being added for easing the understanding of what the transpiler produces. We now see the result is that a tree-like logical structure in CData was flattened into a concatenation of arrays in C that each represent one of the primitive leaves of the original. As for the sanity check mentioned above, we notice that the final struct contains two arrays of 100 floats, while the original contained an array of 100 2-float items, hence the size has been preserved while, at the same time, the data was reorganized. Now, the transpiler must also rewrite the member accesses, and it does so, say in the case of the `particles[1].x - particles.Particle_100_Particle_x_1[1]` transformation, by first deciding which final array a field access maps to, in this case the field access is: array of size 100 of items of type soa Particle - item of type soa Particle - field x of non-array type, so it will map to that field in the final struct that is named Particle_100(from the array)_Particle_x_1(from the type soa Particle and the field x and the non-array type,

hence size 1). As for the index into that array, that is 1, since the individual particle in the CData source was at index 1 hence its primitive fields will be at index 1 in their respective arrays in C; the full formula for index computation is not trivial and will be given and exemplified with more complex soa's to provide better clarity.

A full example of SOA transpilation in CData was presented. Granted, this particular transpilation will now change the execution speed of the program compared to writing a `struct Particle {float x; float y;};` and using a variable `struct Particle var[100]` - in that case there would be no padding inside the struct and this reordering neither adds nor removes padding, so the memory is already in an optimal state from this perspective, but further complex examples and test results will prove the power of SOA over AOS.

3.5 A complex static soa example

Let us now look at what transpilation happens in the case of the nested soa defined and used in Listings 3.3 and 3.4 and who's tree representation is in Figure 3.2.

Listing 3.3: Complex CData static soa sample

```
soa A{
    int b;
    char c[10];
};
soa B{
    soa A d[20];
    int e;
};
int main(){
    soa B f[30];
    f[2].e = 0;
    f[3].d[4].b = 1;
    f[5].d[6].c[7] = 'a';
    return 0;
}
```

Listing 3.4: The transpiled code

```
struct B_30 {
    int B_30_B_d_20_A_b_1[600];
    char B_30_B_d_20_A_c_10[6000];
    int B_30_B_e_1[30];
```

```
};
int main()
{
    struct B_30 f;
    f.B_30_B_e_1[2] = 0;
    f.B_30_B_d_20_A_b_1[(3) * 20 + 4] = 1;
    f.B_30_B_d_20_A_c_10[(5) * 200 + (6) * 10 + 7] = 'a';
    return 0;
}
```

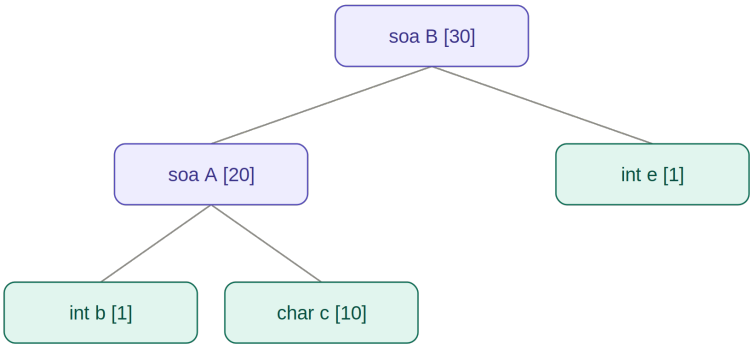


Figure 3.2: The tree representation

We now see a nested soa: `soa B` contains an array of `soa A`, hence the transpiler will recurse into the tree in order to produce the resulting struct. First, the transpiler constructs the list of inorder leaves, which is {int b; char c; int e}, so the struct will have three fields representing these leaves. We now see better the fact that the transpiler takes the tree and flattens it, in the sense that it replaces a soa with its leaves in recursive fashion. The three generated fields will have the data type of array of their original type and their size is computed by multiplying the array sizes from the root of the tree down to the respective leaf: the array corresponding to `char c` will contain 30 * 20 * 10 = 6000 entries, exactly what we see in the resulting C source, and its name will be constructed from this path as well, through concatenation: "B_30" from the `soa B [30]` type of the variable in CData, "B_d_20" because, within `soa B`, the field of type `soa A [20]` is named d, and "A_c_10" because, within the `soa A` we recurse into, the field of type `char [10]` is called c, hence we fully built the recursive path into the resulting variable name. Now, as for indexing into these fields, we arrive at the full recursive formula mentioned before, and we explain it for the same c field in the case of the `f[5].d[6].c[7]` access. We require the seventh entry of the c char array field of the sixth entry of the d `soa A` array field of the

fifth entry of the `soa B` array, but we compute it in reversed order: each entry of the `soa B` array owns a number of (total size of the final array / number of entries in the `soa B` array) = (6000 / 30) = 200 chars, and we want to index into the fifth entry of the `soa B` array, hence we start with the index 5 * 200; then, each entry of a `soa A` array owns (number computed at the previous step / array size at the current level in the tree) = (200 / 20) = 10 chars, and we want to index into the sixth entry of a `soa A` array, hence we add 6 * 10 and currently have index 5 * 200 + 6 * 10; then, each entry of a char array owns (number computed at the previous step / array size at the current level in the tree) = (10 / 10) = 1 char, exactly what we expected since each char should only own one char, and we want to index into the seventh entry of a char array, hence we add the term 7 * 1 and arrive at the final index: [5 * 200 + 6 * 10 + 7 * 1], which is what the transpiler produced as well. The tree structure and data associated with each node helped make both the struct field generation and the index fix-ups simple operations that can both be clearly understood and rapidly performed since most nested structures will likely not have more than a few depth layers, in any case the operations being in θ(n) where n is the depth level of each node.

We may now also compare and contrast the structure produced by the transpiler with one that would have been produced by a C compiler for the equivalent C nested struct whose code is written in Listing 3.5.

Listing 3.5: Equivalent C static nested struct sample

```
struct A2{
    int b2;
    char c2[10];
};
struct B2{
    struct A2 d2[20];
    int e2;
};
int main(){
    struct B2 f2[30];
    f2[2].e2 = 0;
    f2[3].d2[4].b2 = 1;
    f2[5].d2[6].c2[7] = 'a';
    return 0;
}
```

The structure of the transpiled struct is trivial - one array per original leaf, hence one array of 600 ints, one array of 6000 chars and one array of 30 ints. Within each of

the arrays, the C compiler will not introduce any padding: an array is homogenous and does not require padding, all elements are already aligned and start addresses multiples of their own size. As such, we have arrays of (using the most widely-spread values for sizes) 600 * 4 = 2400 bytes, 6000 * 1 = 6000 bytes and 30 * 4 = 120 bytes. The compiler could add padding between the arrays to have them start at addresses multiples of the size of their element type - the first array starts at address 0 within the struct, hence already aligned, the second array starts at address 2400 but has 1-byte wide elements, hence it is aligned no matter where it starts, and the third array starts at address 2400 + 6000 = 8400, which is a multiple of four hence no padding is required there either. The CData system has, thus, managed to construct a memory object with zero padding inside, and this is not incidental - since all the fields of a resulting struct are arrays of primitve data types, the only places where padding may be required in order to have the arrays start at aligned positions is between the arrays, and the maximum possible padding necessary there is the size of the largest primitive in any array minus the size of the smallest primitive in any array, and thus, the bigger the arrays are, the smaller the percentage this padding represents out of the whole memory object [Dre].

The structure of the classic C representation, however, is a different matter entirely. There, the f2 memory object is an array of 30 contiguous struct B2's, each containg 20 contiguous struct A2's and one int, and each of those struct A2's contains one int then 10 contiguous chars. Looking at one struct A2 memory layout, it has one int that starts at address 0, hence is aligned, and 10 chars that are aligned. However, this means one struct A2 is 14 bytes wide, and the compiler needs to guarantee each final leaf is aligned to a memory address a multiple of its size, hence it will add 2 bytes of padding after every struct A2 to ensure the int within struct A2 always starts at a memory address multiple of four when struct A2 is stored in arrays. The memory layout of struct A2 is pictured in Figure 3.3, where B denotes bytes.

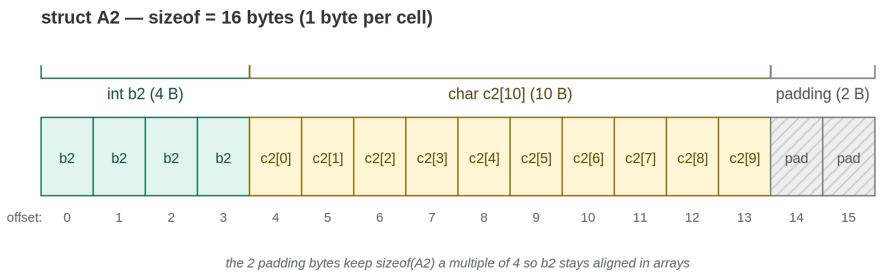


Figure 3.3: The memory layout of a struct A2

At the next level, we have an array of struct A2's, d2, that is comprised of 20

aligned memory objects, however each contains 2 padding bytes, so one d2 memory object contains 40 padding bytes. The 4-byte wide int e2 starts at offset 20 * sizeof(struct A2) = 20 * 16 = 320, hence is aligned. The widest primitive within a struct B2 is an int, and the size of a struct B2 is a multiple of 4, hence struct B2's will always be aligned in arrays and no padding is needed at the end of one. At the topmost level, the struct B2 array f2 contains 20 struct B2's which are all aligned, and that is all - the size of f2 in memory is 20 * sizeof(struct B2) = 30 * 324 = 9720. The size difference from the soa version of this variable is 9720 - 600 * 4 - 6000 * 1 - 30 * 4 = 1200, this coming exclusively from the 2 padding bytes per struct A2 * 20 struct A2's per struct B2 * 30 struct B2's in the array f2, and representing around 12% of the size of the memory object, a significant proportion. This is caused by the fact that CPU's often enforce that every primitive in the memory of a program must start at an address multiple of its own size such that it cannot end up straddling between two cache lines, as explained before, and the compiler makes sure of this by adding padding. This is not an oversight by the designers of C, but a choice made with the goal of easing the understanding of the C memory model: fields in structs appear in memory in the exact order they were declared in. CData does add confusion and uncertainty by turning trees into linear structures, but the complex naming system was added in the hope of lessening the cognitive burden, and, as can be deduced from the examples, the tradeoff between the two approaches is that C offers better readability and easier inference, while CData improves performances by avoiding having useless padding in the cache, thus increasing the proportion of the program's data that fits in every level of the caching system and reducing cache misses due to insufficient cache capacity [BJ19][Dre].

As far as the Odin comparison goes, CData performs much of the same soa transformations that Odin does, but with different use patterns being enforced: in Odin, every nested struct within a composite declaration must be specifically marked with #soa in order to be transformed, otherwise it is left in its (likely padded) classic struct layout, therefore, if the programmer wants to have full soa-ization for all recursive levels of some struct, they must add #soa by hand everywhere, but this also gives them flexibility since they are aware of the usage patterns in the program.[Hal] An example of this, and rather a counterexample to the usefulness of CData, can be iteration: often, iteration is done over all instances of a leaf in a nested structure, for example in a struct with many fields and some price field, it may be the case that for loops target the price field disproportionately more than the other fields, here the SOA pattern improving caching by not putting all the fields of the structs in the cache but rather only the price bytes; on the other hand, in a particle simulation system with a struct Particle with three float components for spatial position, it is to be expected most for loops will touch all three components of a struct

in each increment, hence, if the struct is in SOA layout, the cache will continually swap the three large arrays in and out for every access, thus causing large slowdowns while the memory swaps are performed. Odin gives programmers the tools to handle this, while the implementation of CData described here does not. On the other hand, from a semantic perspective, a positive aspect CData provides is that static soa accesses are identical to C struct accesses, as can be seen in the examples: `f[5].d[6].c[7] = 'a';` in CData and `f2[5].d2[6].c2[7] = 'a';` in C are identical, the semantics of arrays and indexing are the same, and the only syntax element introduced for static soa's is the soa keyword, the rest is classic C [KR78]. One detail worth mentioning is that the described implementation does not handle multilevel arrays, e.g. within a soa it is not legal to have `int a[2][3]`, and does not allow the mixing of structs with soa's, both can be nested with themselves in CData but not between them, not because they would be incompatible but due to the author focusing on implementing other features, as described in the next sections.

3.6 Pointers to soa types

In situations when variablity in the size of data exists, which are encountered often, static allocation does not suffice, this being the reason system like the heap and allocators like C's malloc were introduced - the program can decide at runtime what size of data it needs, an allocator locks a chunk of the heap of the appropriate size, perhaps some function is called to initialize the chunk and format it according to the memory object's needs and that is it. In this direction, CData provides the option to work with pointers to soa types, just like C provides pointers to structs. First, let us exemplify this behaviour and then we will provide the general mechanism, Listing 3.6 provides a basic example and Listing 3.7 provides the transpiled code.

Listing 3.6: Pointer-to-soa usage

```
soa Particle {
    float x;
    float y;
};

int main(void) {
    soa Particle *pool;
    pool = malloc(sizeof(soa Particle) * 100);
    pool@100@[i].x = 1.0;
    return 0;
}
```

Listing 3.7: Pointer-to-soa usage transpiled

```
struct Particle_100 {
    float Particle_100_Particle_x_1[100];
    float Particle_100_Particle_y_1[100];
};
int main()
{
    void *pool;
    pool = malloc(((sizeof(float) + sizeof(float)) * 100));
    ((struct Particle_100 *)pool)->
    Particle_100_Particle_x_1[i] = 1.0;
    return 0;
}
```

The first mechanism necessary to understand the code is the `sizeof(soa)` one, that is, the transpiler scans the syntax tree for the operator sizeof applied to soa types and replaces them with a sizeof expression built from the inorder leaf list corresponding to that soa type, recursively and with coefficients, in this simple example the result being `1 * sizeof(float) + 1 * sizeof(float)`, but if one of them were an array of size n then its sizeof would have a multiplier n, and if one of them was itself a soa type the transpiler would have replaced it with its flattened equivalent. This is the process by which the allocator, in this implementation hardcoded as the malloc allocator, is provided the necessary information about the total size of the memory object it needs to allocate for, although this implementation is fragile since it does not account for padding bytes between the arrays of the transpiled object and will break if any such padding exists. Then, it may be observed that the variable of type `soa Particle *` in CData becomes a `void *` in C, and this happens for any pointer-to-soa type, it is transpiled to a void * and then compile-time pointer arithmetic ensures correct dereferencings of the `void *`. What is interesting is the access pattern for such soa pointers: when a soa memory region is accessed through a pointer, the array size (i.e., number of soa's that region contains) must be mentioned between @ signs, a thing which is not necessary in classic C pointer use. This is both new syntax and new semantics, and for good reason - in C, the size and internal structure of any memory object of any type is known at compile time - it is not possible to encode variable-size structs, their fields and array sizes must be specified exactly, hence at compile time if we wish to encode an access through a pointer to a region holding several structs and each struct has as first field an integer and the access target is the integer of the third struct, we simply encode the access as: access the four bytes starting at (the value of the

pointer + sizeof(the given struct) * 2 + 0), and the fact that all data types are fully determined at compile time ensures this type of pointer arithmetic encoding can be and is always performed at compile time. On the other hand, in CData, the final offset a given field has relative to some address also depends on the number of soa's contained in the chunk it belongs to - if the field is the second field declared in its soa's definition then its array will be the second array in the transpiled struct, but a given instance of a field, say the second field of the third soa logical object, is at offset (base address of the memory chunk + size of the array corresponding to the first field + 2 * the size of said second field), and the size of the array corresponding to the first field depends on the total number of soa logical objects in that chunk of the heap, information which, in our pointer access case, only appears as part of the argument of some previous malloc, information impossible for the transpiler to track. Hence, the programmer must remember, in the use pattern presented, the number of soa's the memory chunk they are indexing into through the pointer contains. The final aspect is the access rewriting itself, which is very similar to the static access case: the transpiler scans the syntax tree, wherever there is an access through a soa pointer it registers the access and generates the struct corresponding to the size mentioned in the `@n@` syntax just like it generated structs of static soa's, it adds a cast to a pointer to that struct type before the `void *` and uses `->` dereferencing, then simply adds an access to the appropriate field by the same mechanism as in the static case, the only novelty was the special void pointer handling. Here, we see the only overhead CData adds to C for implementing the soa functionality, and, while it is not runtime overhead, it does add special semantics - since pointers may be reassigned to point to soa arrays of any size, every access through them must also carry information about the total size of the destination and the programmer must understand this as a change to the C memory model, and the author could not conceive of an implementation that was zero-abstraction for this use case, i.e. they could not find a way to remove the need to mention sizes when accessing pointers. That being said, the semantic ambivalence of pointers as arrays (i.e. a pointer can be directly indexed without being dereferenced first as it is considered it points to an array of its base type) is maintained in CData, as noted in the example.

3.7 Hot-cold splitting

The second addition CData brings is the ability to have the transpiler perform automatic hot-cold splitting.[Dre] This references the temperature of data, i.e. data that is accessed often throughout a program is considered to be hot, and data that is rather not is considered to be cold. Regarding this, there have emerged ideas that it could prove useful to keep hot data close together such that it can fill the cache with-

out having cold data interleaved with it. In the case of CData, the author decided this concept can be applied as follows: C structs contain a number of fields known at compile time; given the program is also known at runtime (this is a simplification and discards dynamic linking), it is possible to design a way to assign, to each variable of a struct, a score representing a rough estimate of the number of times that variable is accessed throughout the program. The framework for computing the score of a field is this: within each function, the scanner may find five statement types that reference the field in some way - reading the field, i.e. the field is part of some rhs expression, writing to the field, i.e. the field is the lhs of some assignment, an if statement that may or may not have the field as part of its comparison, a for statement that may or may not have the field referenced in its initialization and iteration sections, and a while statement that may or may not have the field referenced in its comparison. This is exemplified in Listing 3.8.

Listing 3.8: Statements that might reference a field

```
struct A{int a;}

int main(){
    struct A v;
    v.a = 1; // writing to field a
    int b = v.a; //reading field a
    if(v.a > 0){ // if condition referencing field a
        v.a--;
    }
    else{
        v.a++;
    }
    for(v.a = 0; v.a++; v.a < 10){
    // for statement referencing field a
        v.a++;
    }
    while(v.a > 0){ // while statement referencing field a
        v.a--;
    }
    return 0;
}
```

As far as the actual computation, the score of an individual field is the sum of its scores in all functions in the program, with the main function receiving some greater weight w_main. The score within each function is the sum of the scores of

the field for each statement it appears in in that function - simple reads and writes add *w_read* and *w_write* scores, an if statement adds the scores of the statements in its then branch times some coefficient *c_then* and the scores of its else branch times some *c_else*, a for statement adds the scores of the statements in its block times some *c_for*, and a while statement adds the scores of the statements in its block times some *c_while*, in the example from Listing 3.8 this sum in the case of field a and function main(): *w_write + w_read + w_read + c_then * (w_write) + c_else * (w_write) + w_write + w_write + w_read + c_for * (w_write) + w_read + c_while * (w_write)*, the explanation being: field is written to, field is read from, field is read from in the if condition, the if statement's then branch contains a field write, the if statement's else branch contains a field write, the field is written to in the for initialization block, the field is written to in the for increment block, the field is referenced in the for condition, the for statement's block contains a field write, the field is referenced in the while condition, the while statement's block contains a field write. The full equations for this computation are laid out as Equations (1) through (6), with f representing the field whose score is computed and s representing any statement.

$$\text{score}(f,s) = \begin{cases} w_{\text{read}} & \text{if } s \text{ reads } f \\ w_{\text{write}} & \text{if } s \text{ writes } f \\ S_{\text{if}}(f,s) & \text{if } s \text{ is an if} \\ S_{\text{for}}(f,s) & \text{if } s \text{ is a for} \\ S_{\text{while}}(f,s) & \text{if } s \text{ is a while} \\ 0 & \text{otherwise} \end{cases} \tag{3.1}$$

$$S_{\text{if}}(f,s) = c_{\text{then}} \sum_{s' \in \text{then}(s)} \text{score}(f,s') + c_{\text{else}} \sum_{s' \in \text{else}(s)} \text{score}(f,s') \tag{3.2}$$

$$\begin{aligned} S_{\text{for}}(f,s) = {} & \text{score}(f,\text{cond}(s)) + \text{score}(f,\text{init}(s)) \\ & + \text{score}(f,\text{incr}(s)) + c_{\text{for}} \sum_{s' \in \text{body}(s)} \text{score}(f,s') \end{aligned} \tag{3.3}$$

$$S_{\text{while}}(f,s) = \text{score}(f,\text{cond}(s)) + c_{\text{while}} \sum_{s' \in \text{body}(s)} \text{score}(f,s') \tag{3.4}$$

$$\text{score}_{\text{fn}}(f,F) = \sum_{s \in F} \text{score}(f,s) \tag{3.5}$$

$$\begin{aligned} \text{score}_{\text{total}}(f) = {} & w_{\text{main}} \cdot \text{score}_{\text{fn}}(f,\text{main}) \\ & + \sum_{F \neq \text{main}} \text{score}_{\text{fn}}(f,F) \end{aligned} \tag{3.6}$$

The scanner builds this computation by recursing into blocks where necessary, but it cannot include terms for, for example, function calls, because it might enter an infinite loop for recursive functions. Thus, this heuristic and simple method can

provide a rough estimate for the access count of all fields in a struct, although a large question this work does not answer is - what should all the coefficients be set to? For example, the coefficients for the two branches of an if statements should be 0.5 since, a priori to the running of the program, it is not known what branch will be taken, but if the programmer knows they program in a style in which then branches are taken in larger proportion than else branches, they may set the coefficients themselves accordingly, thus the system gives them the tools to ad-hoc adapt the language to their style.

The next question is - how are these scores used? For each struct (not soa, this system has only been designed for structs) in the CData source, the transpiler checks the scores of all arrays of primitives and, if the score of one of them is less a given proportion, encoded in a `c_min_ratio`, of the sum of the access scores of all the fields, this representing that that field is accessed significantly less than other fields, that field is considered cold data and the transpiler separates it: it is encoded as a pointer to its base type instead of an array of its base type and malloc calls are added after every declaration of variables of the type of the struct the field belongs to, as is demonstrated in Listings 3.9 and 3.10.

Listing 3.9: CData source with both a hot and a cold field

```
struct A{
    int a;
    int b[50];
};

int main(void) {
    struct A v1;
    struct A v2[20];
    for(;;){v1.a = 1;}
    v1.b[0] = 1;
    return 0;
}
```

Listing 3.10: The transpilation of the cold field

```
/*
 * Field access scores:
 *   struct A:
 *     a: 10.50
 *     b: 1.05
 */
struct A {
    int a;
```

```
    int *b;
};
int main()
{
    struct A v1;
    v1.b = malloc(sizeof(int) * 50);
    struct A v2[20];
    for (int __cdata_init_i = 0; __cdata_init_i < 20;
    __cdata_init_i++)
    {
        v2[__cdata_init_i].b = malloc(sizeof(int) * 50);
    }
    for (; ; )
    {
        v1.a = 1;
    }
    v1.b[0] = 1;
    return 0;
}
```

We see the scores of the fields of `struct A`, computed with some given parameter choice, and the field `b` has been considered cold since, in this case, the threshold was 10% and field `b` is accessed less than that: (1.05 + 10.50) * 10% >1.05. Thus, the field of type `int []` was converted to the `int *` type, in the places where its enclosing struct was declared as a singleton a `malloc()` call was injected right after in order to properly initialize the field and in the places where its enclosing struct was declared as an array a for loop of `malloc` running as many iterations as there are array items was injected right after the declaration to properly allocate space on the heap equivalent to what would have been allocated on the stack had the field have been left a pointer. A side effect of putting data away on the heap in this way is that no access rewriting is need: the field was an array type and its elements were accessed by indexing, and the same indexing works identically on the transpiled pointer type due to C's array-pointer ambivalence [KR78].

As for the purpose of all these computations and transformations, it is rather simple - if a struct contains some array that is accessed rarely, that array takes up space in the heap when the struct is brought into heap, space that could be filled up more wisely by other useful, more often-used, in other words, "hotter" data. By turning arrays into pointers, on 64-bit systems, hence the hot-cold splitting could improve performance, at the cost of introducing function calls and runtime management overhead.

Chapter 4

The design and implementation of the application

The application for this work consists of two programs - one transpiler program for the CData language and a minimal text editor that allows for displaying and editing the CData source code in parallel with displaying the resulting C code that results from the transpilation, both created in the C# .NET environment.

4.1 The creation of the transpiler

The transpiler program presented here is standard and follows the settled norms for transpilation.

First, the program receives the textual source representing CData code. This input is lexed by a lexer component and the result is an intermediary representation of the code, that now comprises of lexemes such as identifiers, operators and delimiters. This lexed representation is passed onto a parser component, that performs recursive parsing on the linear token input and constructs a tree, namely a syntax tree, that, at this step, represents the program as a nesting of statements. The output, the syntax tree, is fed into the transpilation component, that parses the tree following predefined rules derived from C's and CData's specification and constructs the equivalent C code to the represented CData code. The most important rules for code conversion, and indeed the only rules that behave differently than in the case of standard C, have been described in the previous chapter, as they relate to the introduced concepts of soa and the hot-cold splitting procedure. The result of this parsing is textual C code - after fully resolving some block from the received syntax tree, the transpiler outputs the corresponding C code. The end result is a textual C source code that adheres to the rules described herein about CData to C conversion.

There are failure states for the transpiler, and the decision was made that any failure is reported immediately to the programmer by the program and the transpilation process stops. Hence, some undefined symbol in the input CData source will cause the lexer component to abort lexing and report the location, within the source stream, of the error. At the next stage, assuming lexing finished without errors, if the parser encounters some undefined token or the structure and disposition of the tokens in the lexer output is, at some location, malformed and unparsable, it reports the location the error first occured at and aborts the parsing process. In the same fashion, if the transpiler component reaches some malformed and untranslateable section, it immediately reports the error location within its input syntax tree and aborts transpilation. The decision for immediate halting was made in order to allow for more time to be allocated to the transpilation algorithms, instead of developing complex debugger and error reporting tools that would not have progressed the main topic of this work.

The development stage for the transpiler consisted of building and testing the lexer component and ensuring it correctly captures all necessary tokens, building and testing the parser component and ensuring it parses its input in correct ways and produces the expected syntax tree for some given input token stream, but also that it integrates correctly with the lexer component and is synchronised with the output rules of the lexing scheme, and building and testing the transpiler component, ensuring it correctly matches and parses the output tree from the parser component, ensuring its rules align with the predefined specification of the CData language and validating that it correctly integrates with the rest of the build system. The concrete testing for the components was performed by validating all primitive operations of each stage, e.g. testing that the parser correctly parses all its prescribed statements in their primitive, i.e. non-nested, forms, then validating increasingly complex and nested examples, until a reasonable of certainty that the system behaves accordingly to the desired specifications was achieved.

4.2 The creation of the text editor

Once the transpiler program was completed, in order to better emphasize its main use case, that of converting one textual CData source into a textual C language source, a minimal text editor was built. It has one use case, wrapping the main use case of the transpiler program - it allows for CData source code to be entered into a text field, allowing for normal facilities such as copying and pasting code and undoing and redoing operations, then, at the user's choice, the transpiler program is invoked on the CData source and the resulting C code is displayed side-by-side. Alternatively, if some error state was reached and the transpilation could not be

completed, the error reported by the transpiler is passed to the user and no resulting C code is displayed.

The development and testing of the textual editor were reduced, mainly thanks to the easily composable GUI features the .NET environment provides. Hence, the two text fields corresponding to the two languages were placed and connected through a minimal backend that invokes the transpiler program, and, if the transpiler reaches some error state, the reported failure information is displayed in a separate section of the editor's main window. Testing was performed by the same procedure as in the case of the transpiler program - examples of CData source code were introduced into the corresponding text field and the ensuing transpilation or error state were validated to align with the expected result and be correctly displayed or reported by the editor.

Chapter 5

Testing and results

In this chapter, the testing CData programs were put through are presented and contrasted to tests on equivalent classic C implementations. As well, similar tests presented in literature are compared to the results herein.

5.1 The testing setup

The machine all reported test were performed on runs the Windows 11 64-bit operating system and is equipped with an i5-13420H Intel processor running nominally at 2.1 GHz, and 16 GB of physical RAM, with an SSD device as the secondary storage. The cache structure is: L1 has 80 KB and is per-core, L2 has 2 MB and is per-core and L3 has 12 MB and is shared between cores. To increase the validity of all tests, the measured programs were all pinned to the same core of the CPU, hence they ran on exactly the same hardware. For the measurement collection, Intel's VTune platform was used [Intc] - it collects data from the performance monitoring unit (PMU) of Intel processors, including the relevant data analysed for the following programs: running time, i.e. the time the program spent on the core, thus excluding the time the operating system put its process to sleep, and memory metrics: L2, L3 and RAM bound time, i.e. the time the processor was stalled while waiting for data to be retrieved from that level, this being interpretable like so: if the L2 bound time is a large proportion of the stall time and L3 and DRAM bound times are low, L1 cache misses were dominant, L2 cache hits were dominant, and L3 and DRAM were either not reached often or cache hits were dominant in their case. More formally, the memory bound times - L1, L2, L3, RAM bounds, are computed by the PMU empirically, i.e. by checking the processor periodically and counting if it is waiting for the return of a request to a memory layer, then, at the end of the reporting period, computing the percentage of time the processor spent waiting in stall mode because of yet unfulfilled requests from each level [Intc]. Given this

framework, we will compare and contrast programs written in CData using the SOA and hot-cold split features, transpiled to C and compiled, with programs written in C in AOS classic style.

5.2 The search space

As far as experimental parameters go, there are the implicit ones, i.e. the chosen languages and the program sources themselves, but there are explicit ones we may tweak as well: in the CData source, we may use SOA programming and hot-cold splitting at the same time or we may not and analyse their effects separately, and, when hot-cold splitting is used, we may tweak any of its field score parameters and, most importantly, the level below which data is considered cold. Of course, the number of times each program is ran in order to produce an average measurement value should also be taken into consideration. Afterwards, the time and memory statistics of these experiments may be plotted in either 2-dimensional graphs, where we fluctuate one single parameter and observe the change in the statistics, or in 3-dimensional graphs, where we fluctuate a pair of parameters and observe the change in the statistics.

The choice was made as follows: 2 programs are to be tested, both written in the two languages. One of them was chosen to emphasize the strengths of data-oriented programming - a particle simulation, a very data-intensive task that comprises of a main loop that repeats continuously, at each iteration going over a large number of memory objects, totaling around 1 gigabyte (GB), and performing small compute operations on each. The other one was chosen to perhaps show a scenario where CData performs the same or slower than C - a linear numerical computation that is not data-intensive, but rather compute-heavy, that comprises of a continuously repeating main loop that performs numerical computation techniques and only touches around 10 kilobytes (KB) of memory, most of it being the program's instructions, hence, the majority of the time the program is expected to be performing computations not requesting memory.

In test number 1, the CData program will be written using both soa-ization and hot-cold splitting with a cold threshold of 10%, the CPU time and memory statistics of these two programs will be computed after running each for 5 times and averaging the results, and they will be reported in a table and contrasted.

In test number 2, we will fluctuate the cold threshold and the coefficient of for statements for computing field scores and compare the runtime statistics obtined for the CData resulting programs, and we will have a 3-dimensional graph only analysing CData results.

In test number 3, we will eliminate hot-cold splitting entirely, only use soaization for the CData source and compare to the C result.

5.3 Test 1 - full CData utilization versus C

We see in Table 5.1 that, in the case of a program representing a particle simulation written in both languages while CData's features are fully utilized, there is an 8% speedup in the total average run time for the CData program, phenomenon possibly explain by better utilization of the cache. This hypothesis is strengthened by the fact that all cache metrics are better in the case of the CData program - it spends 2.5% less of its clock cycles waiting for RAM requests, 1.2% less waiting for L3 and 0.3% less waiting for L2 than the same program written in C, and, although these differences are not even in the order of 10%, the runtime indicates that they might compound to a tangible speedup, hence the implemented abstractions of CData might have value.

Metric	CData Program	C Program
L2 Bound	0.1%	0.4%
L3 Bound	0.2%	1.4%
RAM Bound	1.6%	4.1%
CPU Run Time	101 s	110 s

Table 5.1: Test 1 results for particle simulation

In the case of the numerical computation, the story is wholly different - Table 5.2 shows the time percentage spent waiting for cache data due to cache misses in higher levels is almost the same for both programs and their runtime is almost the same, CData even produced a program that ran by one second slower. This is to be expected, since the numerical touched only a few KB of data that comfortably fit within the L1 cache and thus is unlikely to benefit from the data rearrangements CData provides, while the particle simulation touched around 1 GB data, definetly overflowing all caches by a large amount (1 GB is significantly more than the caching system's 80 KB + 2 MB + 12 MB) and benefitting from useful data staying in the cache for longer.

5.4 Test 2 - tweaking hot-cold parameters

Although the hot-cold splitting systems contains many parameters and an extensive search through their space could provide interesting insights, for this test the author selected those that seemed the most impactful - the cold threshold that

Metric	CData Program	C Program
L2 Bound	0.2%	0.2%
L3 Bound	0.2%	0.3%
RAM Bound	1.2%	1.1%
CPU Run Time	68 s	67 s

Table 5.2: Test 1 results for numerical computation

determines below which frequency an array is turned into a pointer, and the for coefficient, which estimates heuristically the number of times a for loop will iterate. Four points in this 2-coordinate space were chosen: (0.1, 10), (0.1, 100), (0.5, 10), (0.5, 100) and the runtimes of the previous CData programs were plotted with these parameter configurations, then a linear surface was interpolated between the 3-dimensional points.

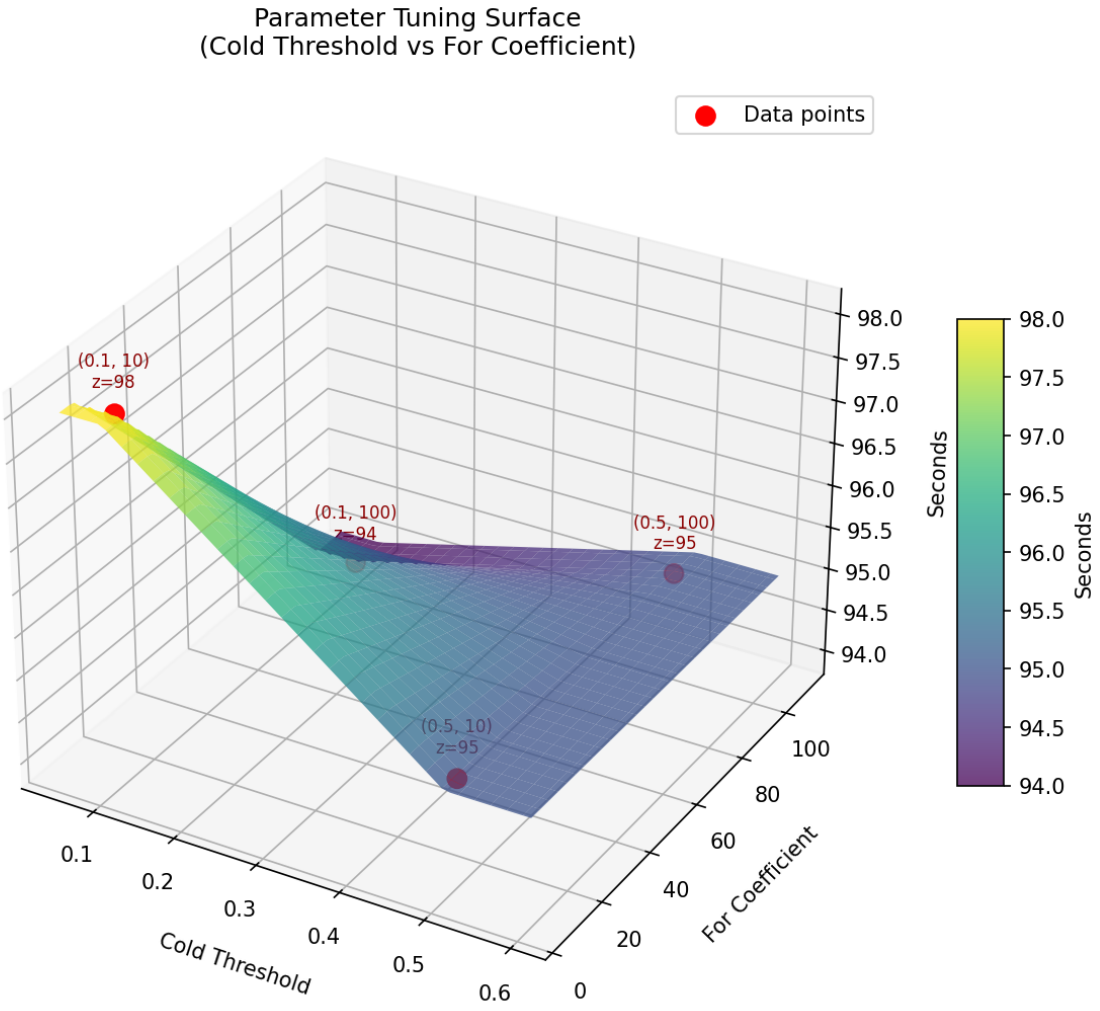


Figure 5.1: Test 2 results for particle simulation

For the particle simulation program, whose results are depicted in Figure 5.1, we

notice changes in the runtime, but not interpretable as linear relations. The parameters used for test 1 for this program prove suboptimal - if for's coefficient was to be increased to 100 or the cold threshold was to be increased to 50% the runtime would decrease by a few percentage points, but if both changes were to happen the result would be the same as for just the threshold increase and worse than the case just with the coefficient increase. This indicates the shape of the parameter space of the hot-cold system is not easily predicted and requires further exploration to enlarge the efficiency increase due to this system.

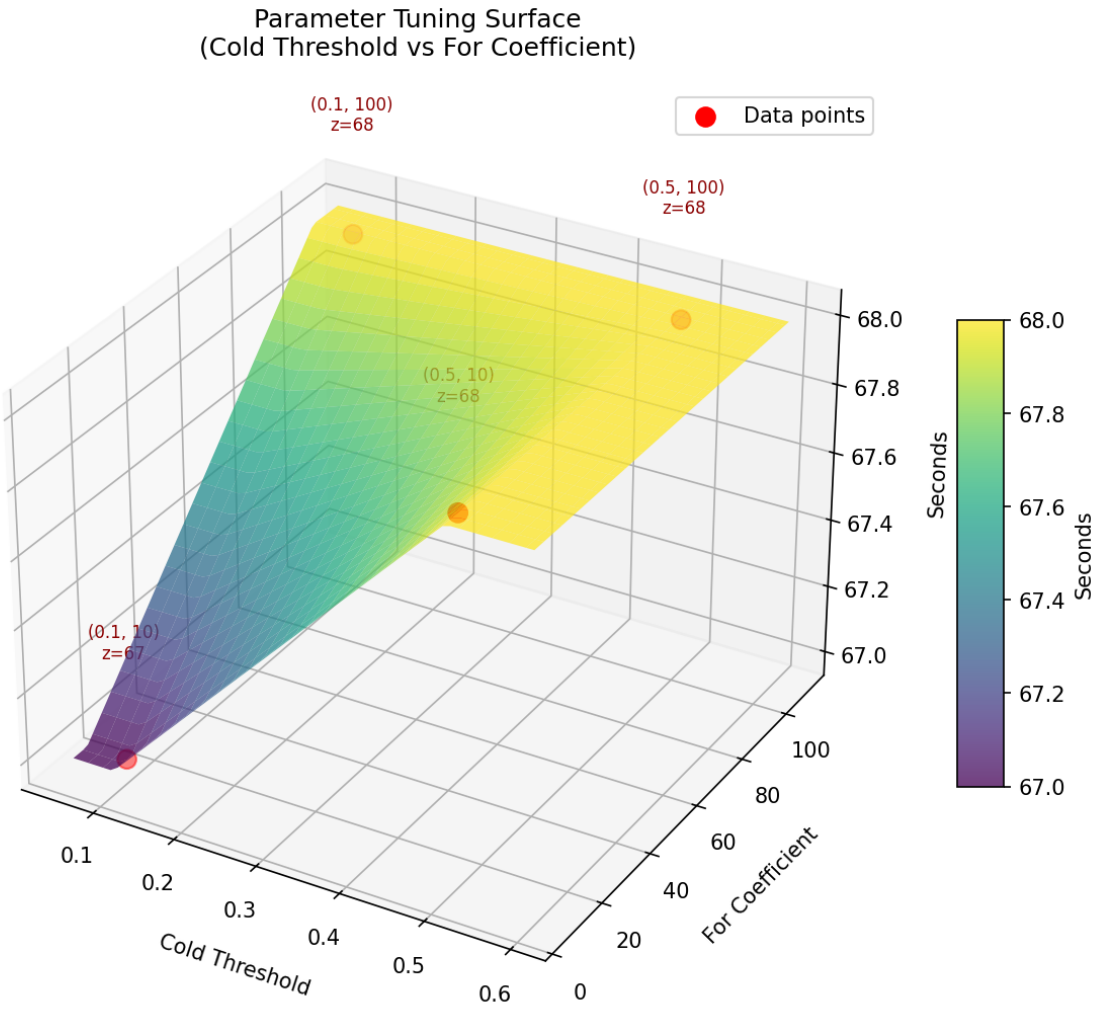


Figure 5.2: Test 2 results for numerical calculation

As for the same parameter exploration applied to the CData numerical computation program, the results are similar to test 1 and are displayed in Figure 5.2 - the changes are small and should probably be interpreted as perturbations in the experimental setup, the reason for this being the same, that the second program was chosen to show that CData only provides meaningful speedups in data-intensive

tasks.

5.5 Test 3 - CData soa utilization versus C

When it comes to removing the hot-cold splitting ability from CData, we arrive at an interesting result, displayed in Table 5.3 - the CData program still runs a few percentage points faster than the C program, but that margin has shrunk visibly, and some of the memory metrics are not as favourable for the CData program as they were in test 1 where hot-cold splitting was used. This indicates that the both the soa system and the hot-cold system provided improvements to the bare C code, enforcing the hypothesis that exploring data transformations at the lower levels of programs might have benefits.

Metric	CData Program	C Program
L2 Bound	0.4%	0.3%
L3 Bound	0.1%	1.4%
RAM Bound	1.9%	4.2%
CPU Run Time	106 s	109 s

Table 5.3: Test 3 results for particle simulation

Metric	CData Program	C Program
L2 Bound	0.1%	0.1%
L3 Bound	0.2%	0.3%
RAM Bound	1.2%	1.1%
CPU Run Time	67 s	67 s

Table 5.4: Test 3 results for numerical computation

As for the numerical simulation now performed without hot-cold splitting, the results follow those for tests 1 and 2 - since not much of the caching system is used anyways, losing data reorderings does not influence run time by a large margin, and the programs written in the two languages behave almost identically both in terms of run time and in terms of cache holdups, the data being presented in Table 5.4.

5.6 Comparison to other DOD tests

In the literature, DOD practices have been tested in similar scenarios, with mixed results. Fontana et al. [FAN+17] demonstrate a K-means optimization algorithm in both DOD- and OOP-style, and the result leaned in the favour of the DOD implementation, in both the single- and multithreaded case, both with regards to runtime

and cache hit percentage, in the same way the test presented here indicate. Arantes et al. [ALB+25] present an intelligent maze solving algorithm was implemented in both DOD- and OOP-style, in single- and multithreaded fashion. However, the results in this case were only marginally better for the DOD implementations, if that, suggesting DOD might not be able to improve run times of general programs no matter the specifics, as the previously presented tests designed to not favour DOD seem to show. Wingqvist et al. [DFS22] performed an experiment where a 2-dimensional Snake-like game was implemented in DOD and OOD (object-oriented design) style, again in both single- and multithreaded fashion, and in this case the results heavily favoured the DOD implementation, this implementation even yielding run time improvements in the order of a 10 times reduction.

Chapter 6

Conclusions

Throughout this work, the idea that data matters above all has been proposed, exemplified, implemented and tested. The history of abstraction and of hardware was invoked in order to set the context of today's dominant software philosophy, object-oriented programming, and an alternative was discussed in great detail - data-oriented design, a collection of principles that guide developers to better understand and reason about the data their programs work with, instead of creating generic solutions that may run on all machines but do not perform up to modern standards on any of them. A new language was proposed, CData, that brings minor semantic and syntactic changes to the classic C implementation in order to give programmers more power to utilize the resources at hand. This language was described and displayed, both in its strengths and weaknesses, and tested on real use cases, both on those that favoured it and those that did not, and it may thus be considered worthy of being trialed and explored further. Interesting ways CData could be extended are, for example, allowing for a system that can mix soa and struct declarations and optimize the resulting memory objects, similar to the one in the implementation of Odin, or the addition of an operating system collaboration system, where the transpiler queries for all available OS information about the hardware of the machine it's running on and optimizes the structure of memory objects to run optimally on that exact machine, adding an important layer of hardware-awareness to the resulting code.

Bibliography

[Act] Mike Acton. Data-oriented design and c++. Insomniac Games, 2014. Link: https://neil3d.github.io/assets/img/ecs/DOD-Cpp.pdf, accessed on 11th of February 2026.

[ALB+25] Gabriel M. Arantes, Giancarlo Lucca, Eduardo N. Borges, Richard F. Pinto, Bruno L. Dalmazo, and Rafael A. Berri. Impact of data-oriented and object-oriented design on performance and cache utilization with artificial intelligence algorithms in multi-threaded cpus. *Revista Junior de Iniciação Científica em Ciências Exatas e Engenharia*, 26(1), 2025.

[Bay22] Jessica D. Bayliss. Developing games with data-oriented design. In *GAS '22: Proceedings of the 6th International ICSE Workshop on Games and Software Engineering: Engineering Fun, Inspiration, and Motivation*, ICSE '22. ACM, 2022.

[BJ19] Luna Backes and Daniel A. Jimenez. The impact of cache inclusion policies on cache management techniques. In *Proceedings of the International Symposium on Memory Systems (MEMSYS '19)*, pages 428–438. ACM, 2019.

[Car02] Carlos Carvalho. The gap between processor and memory speeds. *ICCA - International Convention for Control and Automation*, 2002. Link: http://gec.di.uminho.pt/discip/minf/ac0102/1000Gap_Proc-Mem_Speed.pdf, accessed on 15th of March 2026.

[DFS22] Wingqvist D., Wickström F., and Memeti S. Evaluating the performance of object-oriented and data-oriented design with multithreading in game development. In *2022 IEEE Games, Entertainment, Media Conference (GEM)*, ICSE '12, page 6. IEEE, 2022.

[Dre] Ulrich Drepper. What every programmer should know about memory. Red Hat, Inc., 2007. Link: https://people.freebsd.org/ lstewart/articles/cpumemory.pdf, accessed on 15th of February 2026.

[Fab18] Richard Fabian. *Data-Oriented Design*. Richard Fabian, USA, 1st edition, 2018.

[FAN+17] Tiago Augusto Fontana, Sheiny Almeida, Renan Netto, Vinicius Livramento, Chrystian Guth Laércio Pilla, and José Luís Güntzel. Exploiting cache locality to speedup register clustering. In *Proceedings of 30th Symposium on Integrated Circuits and Systems Design*. IEEE, 2017.

[Fog] Agnes Fog. Lists of instruction latencies, throughputs and micro-operation breakdowns for intel, amd, and via cpus. Link: https://www.agner.org/optimize/instruction_tables.pdf, accessed on 19th of March 2026.

[FP19] Edoardo Carra F. Pellacini, Giacomo Nazzaro. Yocto/gl: A data-oriented library for physically-based graphics. *STAG: Smart Tools and Applications in Graphics*, 2019. Link: https://diglib.eg.org/items/7587e350-9350-4f63-95d2-5420a5037351, accessed on 10th of March 2026.

[Hal] Bill Hall. odin-lang. https://github.com/odin-lang. official Odin language implementation source, accessed on 10th of March 2026.

[Inta] Intel. Intel 64 and ia-32 architectures software developer's manual. Link: https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html, accessed on 10th of March 2026.

[Intb] Intel. Intel processor datasheet 1 of 2 - 13th generation intel core. Link: https://www.intel.com/content/www/us/en/content-details/743844/13th-generation-intel-core-intel-core-14th-generation-intel-core-processor-series-1-and-series-2-and-intel-xeon-e-2400-processor-datasheet-volume-1-of-2.html, accessed on 21st of April 2026.

[Intc] Intel. Intel vtune profiler documentation. Link: https://www.intel.com/content/www/us/en/developer/tools/oneapi/vtune-profiler-documentation.html, accessed on 11th of April 2026.

[KR78] Brian Kernighan and Dennis Ritchie. *The C Programming Language*. Prentice-Hall, USA, 1st edition, 1978.

[MWN22] Suzanne J. Matthews, Kevin C. Webb, and Tia Newhall. *Dive Into Systems: A Gentle Introduction to Computer Systems*. No Starch Press, USA, 1st edition, 2022. Link: https://diveintosystems.org/, accessed on 11th of February 2026.

[Nyb21] Frank Nyberg. Investigating the effect of implementing data-oriented design principles on performance and cache utilization. Master's thesis, UMEÅ UNIVERSITY, Faculty of Science and Technology, Department of Computing Science, 7 2021. Link: https://umu.diva-portal.org/smash/record.jsf?pid=diva2%3A1578616dswid=-629, accessed on 21st of March 2026.

[O'R16] Gerard O'Regan. *Introduction to the History of Computing: A Computing History Primer*. Springer, USA, 1st edition, 2016.

[PH11] David Patterson and John Hennessy. *Computer Organization and Design: The Hardware/Software Interface*. Morgan Kaufmann, USA, 4th edition, 2011.

[Sha48] Claude Shannon. *Dive Into Systems: A Gentle Introduction to Computer Systems*. Nokia Bell Labs, USA, 1st edition, 1948. Link: https://ieeexplore.ieee.org/document/6773024, accessed on 12th of February 2026.

[Str] Bjarne Stroustroup. What is 'object-oriented programming'? ATT Bell Laboratories, 1991. Link: https://www.stroustrup.com/whatis.pdf, accessed on 22nd of March 2026.

[Tan14] Andrew Tannenbaum. *Modern Operating Systems*. Pearson, USA, 1st edition, 2014.

[UoP] USA University of Pittsburgh. *Presentation of the MESI protocol*. USA, 1st edition. Link: https://people.cs.pitt.edu/ melhem/courses/2410p/ch5-4.pdf, accessed on 12th of February 2026.

[vN] John von Neumann. First draft of a report on the edvac. Link: https://ieeexplore.ieee.org/document/238389, accessed on 10th of February 2026.